

IMAT2024 Matematikk for ingeniørfag 2D

Gruppe 4

Kandidatnummer: KAND1, KAND2, KAND3, KAND4, KAND5

Gruppeprosjekt

Vår 2025

Innhold

1	Innledning	4
2	Del 1: Trafikkmodell: tetthet, hastighet og fluks	5
2.1	Oppgave 1	5
2.1.1	a) Funksjonen $u(x, t)$	5
2.1.2	b) Funksjonen $v = v(u)$	5
2.1.3	c) Den deriverte	14
2.1.4	d) Utregning av fluksen	19
2.1.5	e) Maksimal fluks	20
2.1.6	f) Differensiallikningen	24
2.2	Oppgave 2	25
2.2.1	a) Initialverdibetingelsen	25
2.2.2	b) Verdiene v_{max} og u_{max} og randbetingelsen for $x = 1000$	26
2.2.3	c) Løs differensiallikningen	26
2.2.4	d) Bevegelsen til en enkelt bil	29
2.2.5	e) Hastigheten til en kø	32
2.2.6	f) Valg av parameter p	34
3	Del 2: Varmeledning	35
3.1	Oppgave 3: Poissonligning, 1D	35
3.1.1	Numerisk løsning av oppgave 3	36
3.2	Oppgave 4: Varmeligning, 1D	37
3.3	Oppgave 5: Poissonligning, 2D	39
3.4	Oppgave 6: Varmeligning	40
3.4.1	a)	40
3.4.2	b)	40
3.4.3	c)	42
3.4.4	d)	44
3.5	Oppgave 7: Luften skal med	45
3.5.1	a)	45
3.5.2	b)	47
3.5.3	c)	48
3.6	Oppgave 8: Oppsummering av gruppearbeidet	50
4	Vedlegg A: Systematiserte kodeblokker til oppgavene	52
4.1	Del 1	52

4.1.1	Oppgave 1b-iii	52
4.1.2	Oppgave 1b-iii	53
4.1.3	Oppgave 2c	54
4.1.4	Oppgave 2e	55
4.2	Del 2	56
4.2.1	Oppgave 3	56
4.2.2	Oppgave 4	57
4.2.3	Oppgave 5	58
4.2.4	Oppgave 6b	59
4.2.5	Oppgave 6c	60
4.2.6	Oppgave 6d	61
4.2.7	Oppgave 7a	62
4.2.8	Oppgave 7b	63
4.2.9	Oppgave 7c	64

Figurer

1	Oppgave 1b-iii: Plott av en logaritmisk funksjon.	10
2	Oppgave 1b-iii: Plott av cosinus-funksjonen.	13
3	Oppgave 1b-iii: Logaritmisk og cosinus-funksjon i samme plott.	14
4	Oppgave 2a: Skisse for en graf for de gitte initialverdiene	25
5	Tetthetsgraf for $t = 35.10s$	29
6	Tetthetsgraf for $t = 200.25s$	29
7	Oppgave 2d: Posisjon og hastighetsgrafer for den bakerste bilen	32
8	Oppgave 2e: Posisjon til bilen som begynner å bevege seg ved tid t	34
9	Oppgave 3: Sammenligning av A. og N. løsning	37
10	Oppgave 4: Varmeligningen med ulike T-verdier for å vise varmefordeling over tid	38
11	Oppgave 5: Poissonligning, 2D, visualisert	40
12	Oppgave 6b: Temperaturfordeling i en gjenstand over tid.	41
13	Oppgave 6c: Temperaturfordeling ved $t \approx 4.7$ minutt	43
14	Oppgave 7: Varmeplot av temperaturfordelingen i legemet og luftlaget	46
15	Oppgave 7: Varmeplot som viser tidspunktet hvor midten av legemet når 60°C	49

Tabeller

1	Oppgave 1c: Oversikt over deriverte, måleenheter og forklaringer.	16
2	Oppgave 1c: Oppsummering av hvordan de deriverte påvirker trafikken.	19

1 Innledning

Denne rapporten tar for seg prosjektoppgaven i Matematikk for ingeniørfag 2D, våren 2025. I dette prosjektet har vi analysert trafikkflyt og varmeledning ved hjelp av numeriske metoder og differensiallikninger.

I første del analyserte vi en trafikkmodell, og beskrev bilbevegelse som en kontinuerlig strøm ved hjelp av ulike hastighetsmodeller og en bevaringslov. Differensiallikningen ble løst numerisk med *Lax-Friedrichs*-metoden, for å simulere trafikkflyten under ulike betingelser, blant annet kødannelse og oppløsning.

I andre del bruke vi varmelikningen til å modellere temperaturfordelingen i et objekt over tid. Numeriske metoder ble benyttet for å løse likningen og analysere varmeutviklingen. Vi undersøkte hvordan randbetingelser og materialegenskaper påvirker varmeoverføringen, samt hvordan et luftlag rundt objektet endrer varmefordelingen.

All kode er inkludert under de respektive oppgavene, men kan også finnes i Vedlegg A, hvor kodeblokkene er formatert for å være mer lesbare og enklere å kopiere for kjøring og testing.

2 Del 1: Trafikkmodell: tetthet, hastighet og fluks

2.1 Oppgave 1

2.1.1 a) Funksjonen $u(x, t)$

i) Hvis tidspunktet $t = t_0$ holdes fast, og vi ser på grafen til $u(x, t_0)$ beskriver grafen hvordan biltettheten varierer langs veienstrekningen på et gitt tidspunkt t_0 . Dette tilsvarer et øyeblikksbilde av trafikken, der x-aksen viser posisjonen langs veien og y-aksen viser tettheten til trafikken ved hver av disse posisjonene. Høye verdier på grafen indikerer områder med høy tetthet (mange biler), mens lave verdier indikerer områder med lav tetthet (få biler).

ii) Om vi holder posisjonen $x = x_0$ fast, og ser på grafen til $u(x_0, t)$, beskriver grafen hvordan biltettheten ved *et spesifikt punkt* langs veien endrer seg over tid. Altså viser grafen tettheten (antall biler per meter) ved posisjon x_0 som en funksjon av tiden t . Høye verdier indikerer perioder med høy biltetthet ved punktet x_0 , mens lave verdier indikerer perioder med lav biltetthet ved x_0 .

iii) Når vi endrer lengden på strekningen R , som brukes til å definere $u(x, t)$, har det innvirkning på hvordan tettheten beregnes og hvor detaljert bilde vi får av trafikken:

- Liten R (f.eks én billengde): Tettheten $u(x, t)$ blir svært lokal og mer sensitiv for små variasjoner, ettersom vi ser på færre biler om gangen. Dette gir et detaljert, men mer støyende bilde, der små endringer i bilfordelingen gir store utslag i tettheten. I numeriske simuleringer kan dette føre til raske svingninger i tetthetsverdiene, spesielt hvis bilene ikke er jevnt fordelt.
- Stor R (f.eks 100 billengder): Tettheten $u(x, t)$ blir en mer jevnet ut gjennomsnittlig verdi over en større veistrekning. Dette gir en mer stabil og helhetlig oversikt over trafikkflyten, men små detaljer forsvinner. Variasjoner i tetthet blir glattet ut, noe som kan være nyttig for å identifisere generelle trender i trafikk tettheten, men kan skjule lokale fenomener som kødannelse.

2.1.2 b) Funksjonen $v = v(u)$

i) Vi ser på antagelsene $v(0) = v_{max}$, $v(u_{max}) = 0$ og $\frac{dv}{du} \leq 0$ punktvis:

1. $v(0) = v_{max}$:

- v er en funksjon av u , $v(u)$, der u er tettheten til bilene.
- Når tettheten $u = 0$, betyr det at det ikke er noen biler på veien, og bilene kan da kjøre med maksimal tillatt hastighet v_{max} .

2. $v(u_{max}) = 0$:

- Når tettheten u er på sitt maksimale, u_{max} , betyr det at veien er full av biler, f.eks. i en trafikkork eller rushtrafikk.
- Ingen biler kan bevege seg, og da blir hastigheten 0.

3. $\frac{dv}{du} \leq 0$:

- Antagelsen sier at hastigheten avtar når tettheten øker.
- Dette er rimelig, siden jo flere biler det er på veien, jo saktere må bilene kjøre for å unngå kollisjoner og forholde seg til de andre trafikkantene.

ii)

Regn og vis at $v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}}\right)^p\right)$ for alle verdier $p \geq 1$ tilfredsstiller de fem antakelsene vi har listet opp ovenfor for hastighetsfunksjonen.

Antagelsene for hastighetsfunksjonen v , som gitt i oppgaven:

1. $v \geq 0$
2. $v \leq v_{max}$, hvor v_{max} er maksfart, dvs. fartsgrensen på veien.
3. $\frac{dv}{du} \leq 0$.
4. $v(0) = v_{max}$ når tettheten $u = 0$.
5. $v(u_{max}) = 0$ når tettheten $u = u_{max}$ er størst mulig.

1. Antagelse: $v \geq 0$:

- Siden u representerer tettet, må $u \geq 0$.
- Vi har også at $u \leq u_{max}$ som betyr at $\frac{u}{u_{max}} \leq 1$.
- Dermed er $\left(\frac{u}{u_{max}}\right)^p \leq 1$ for alle $p \geq 1$.
- Dette medfører at $1 - \left(\frac{u}{u_{max}}\right)^p \geq 0$
- Siden v_{max} er positiv (maksimal hastighet), er $v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}}\right)^p\right) \geq 0$.

2. Antagelse: $v \leq v_{max}$, hvor v_{max} er maksfart:

- Fra antagelse 1 vet vi at $1 - \left(\frac{u}{u_{max}}\right)^p \leq 1$
- Dermed er $v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}}\right)^p\right) \leq v_{max}$

3. Antagelse: $\frac{dv}{du} \leq 0$:

Vi deriverer $v(u)$ med hensyn på u :

$$\begin{aligned}\frac{dv}{du} &= v_{max} \cdot \left(-p \cdot \left(\frac{u}{u_{max}}\right)^{p-1} \cdot \frac{1}{u_{max}}\right) \\ &= v_{max} \cdot \left(-p \cdot \frac{u^{p-1}}{u_{max}^{p-1}} \cdot \frac{1}{u_{max}}\right) \\ &= -p \cdot v_{max} \cdot \frac{u^{p-1}}{u_{max}^{p-1} \cdot u_{max}} \\ &= -p \cdot v_{max} \cdot \frac{u^{p-1}}{u_{max}^p} \\ &= -\frac{p \cdot v_{max}}{u_{max}^p} \cdot u^{p-1}\end{aligned}$$

- Vi vet at v_{max} (maksimal hastighet), u_{max} (maksimal tetthet) per definisjon er positive, og $p \geq 1$ er gitt i oppgaven.
- u^{p-1} er positiv. Dette fordi u er ikke-negativ, og en ikke-negativ verdi opphøyd i en eksponent som er større enn eller lik null vil alltid være ikke-negativ. Oppgaven gir at $p \geq 1$, dermed vet vi at eksponenten $(p-1) \geq 0$.
- Når alle faktorene i uttrykket for $\frac{dv}{du}$ er positive eller null, vil minustegnet si at $\frac{dv}{du} \leq 0$.
- Dette betyr at hastigheten avtar, eller er konstant, når tettheten øker.

4. Antagelse: $v(0) = v_{max}$ når tettheten $u = 0$:

Setter vi inn $u = 0$ i funksjonen, får vi:

$$\begin{aligned}v(u) &= v_{max} \cdot \left(1 - \left(\frac{u}{u_{max}}\right)^p\right) \\v(0) &= v_{max} \cdot \left(1 - \left(\frac{0}{u_{max}}\right)^p\right) \\&= v_{max} \cdot (1 - 0) \\&= v_{max}\end{aligned}$$

5. Antagelse: $v(u_{max}) = 0$ når tettheten $u = u_{max}$ er størst mulig:

Setter vi $u = u_{max}$ inn i funksjonen får vi:

$$\begin{aligned}v(u) &= v_{max} \cdot \left(1 - \left(\frac{u}{u_{max}}\right)^p\right) \\v(u_{max}) &= v_{max} \cdot \left(1 - \left(\frac{u_{max}}{u_{max}}\right)^p\right) \\&= v_{max} \cdot (1 - 1^p) \\&= v_{max} \cdot (1 - 1) \\&= v_{max} \cdot 0 \\&= 0\end{aligned}$$

Vi ser at når tettheten når maksimal verdi, vil det resultere i at hastigheten når nullpunkt.

iii) Mulige andre funksjoner som tilfredsstiller antagelsene, kan være en logaritmisk funksjon og cosinus-funksjon.

Logaritmisk funksjon:

$$v(u) = \begin{cases} v_{max} \cdot \frac{1 + \ln\left(1 - \frac{u}{u_{max}}\right)}{1 + \ln(2)} & \text{når } 0 \leq u < u_{max} \\ 0 & \text{når } u = u_{max} \end{cases}$$

Cosinus-funksjon:

$$v(u) = v_{max} \cdot \cos\left(\frac{\pi}{2} \cdot \frac{u}{u_{max}}\right)$$

Hvorfor disse funksjonene tilfredsstiller antakelsene:

Logaritmisk funksjon

$$v(u) = \begin{cases} v_{max} \cdot \frac{1 + \ln\left(1 - \frac{u}{u_{max}}\right)}{1 + \ln(2)} & \text{når } 0 \leq u < u_{max} \\ 0 & \text{når } u = u_{max} \end{cases}$$

1. Antagelse: $v \geq 0$:

For $u \leq u < u_{max}$ har vi at:

$$0 < \left(1 - \frac{u}{u_{max}}\right) \leq 1$$

La $x = 1 - \frac{u}{u_{max}}$.

Da har vi at $0 \leq u < u_{max}$.

Vi ser på funksjonen $f(x) = 1 + \ln(x)$ for $0 < x \leq 1$.

Grenseverdi:

$$\lim_{x \rightarrow 0^+} (1 + \ln(x)) = -\infty$$

Derivert:

$$f'(x) = \frac{1}{x}$$

Siden $x > 0$, er $f'(x) > 0$. Altså er $f(x)$ strengt voksende på intervallet $(0, 1]$.

Verdi ved $x = 1$:

$$\begin{aligned} f(1) &= 1 + \ln(1) \\ &= 1 + 0 \\ &= 1 \end{aligned}$$

Siden $f(x)$ er strengt voksende ($f'(x) > 0$) og kontinuerlig på intervallet $(0, 1]$, og $f(1) = 1$, så er $f(x) \geq 0$ for alle x i $(0, 1]$. Dermed er telleren, $1 + \ln(1 - \frac{u}{u_{max}}) \geq 0$, altså alltid positiv. Siden nevneren $(1 + \ln(2))$ og v_{max} også er positive, følger det at $v(u) \geq 0$.

2. Antagelse: $v \leq v_{max}$, hvor v_{max} er maksfart:

Som vist over, er $1 + \ln\left(1 - \frac{u}{u_{max}}\right) \leq 1$. Og siden nevneren, $1 + \ln(2)$, er større enn 1, må hele brøken,

$$\frac{1 + \ln\left(1 - \frac{u}{u_{max}}\right)}{1 + \ln(2)}$$

være mindre enn eller lik 1. Multiplisert med v_{max} gir dette $v(u) \leq v_{max}$.

3. Antagelse: $\frac{dv}{du} \leq 0$:

Vi deriverer $v(u)$ med hensyn på u :

$$\begin{aligned} \frac{dv}{du} &= \frac{v_{max}}{1 + \ln(2)} \cdot \frac{d}{du} \left(1 + \ln\left(1 - \frac{u}{u_{max}}\right)\right) \\ &= \frac{v_{max}}{1 + \ln(2)} \cdot \left(\frac{1}{1 - \frac{u}{u_{max}}}\right) \cdot \left(-\frac{1}{u_{max}}\right) \\ &= -\frac{v_{max}}{(1 + \ln(2))} \cdot u_{max} \cdot \left(1 - \frac{u}{u_{max}}\right) \end{aligned}$$

Siden $0 \leq u < u_{max}$, er $1 - \frac{u}{u_{max}} > 0$.

v_{max} , u_{max} og $(1 + \ln(2))$ er positive konstanter.

Dermed er nevneren positiv, og minustegnet gjør hele uttrykket negativt.

Altså er $\frac{dv}{du} < 0$.

4. Antagelse: $v(0) = v_{max}$ **når tettheten $u = 0$:**

For å undersøke antakelsen, setter vi $u = 0$ inn i funksjonsuttrykket $v(u)$ og sjekker om resultatet blir v_{max} .

$$\begin{aligned} v(0) &= v_{max} \cdot \left(\frac{1 + \ln\left(1 - \frac{0}{u_{max}}\right)}{1 + \ln(2)} \right) \\ &= v_{max} \cdot \left(\frac{1 + \ln(1)}{1 + \ln(2)} \right) \\ &= v_{max} \cdot \left(\frac{1 + 0}{1 + \ln(2)} \right) \\ &= v_{max} \cdot \left(\frac{1}{1 + \ln(2)} \right) \cdot (1 + \ln(2)) \\ &= v_{max} \end{aligned}$$

Vi ser at $v(0) = v_{max}$.

5. Antagelse: $v(u_{max}) = 0$ **når tettheten $u = u_{max}$ er størst mulig:**

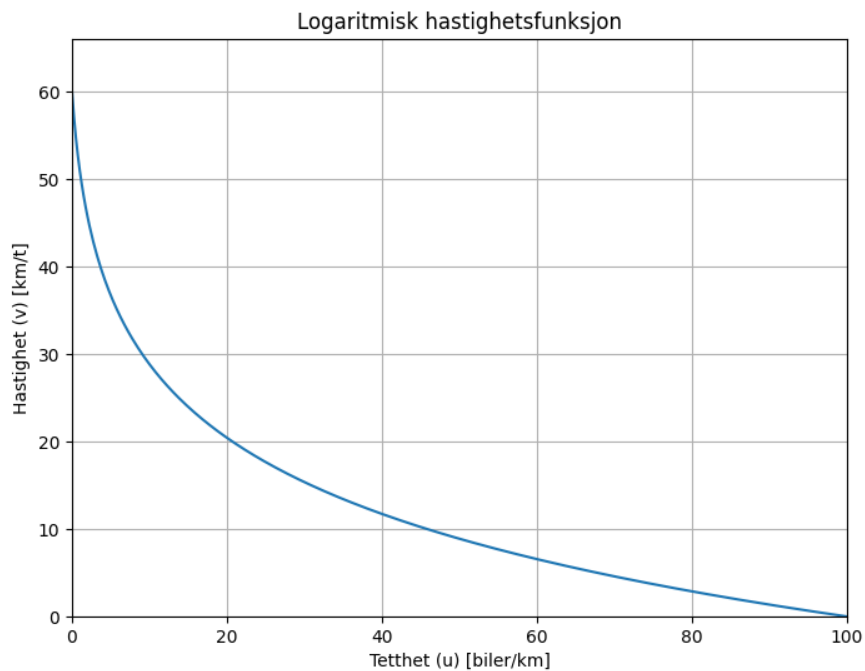
Denne antagelsen er definert i det foreslåtte funksjonsuttrykket:

$$v(u) = \begin{cases} v_{max} \cdot \frac{1 + \ln\left(1 - \frac{u}{u_{max}}\right)}{1 + \ln(2)} & \text{når } 0 \leq u < u_{max} \\ 0 & \text{når } u = u_{max} \end{cases}$$

Årsaken er at ved maksimal tetthet, u_{max} er veien fullpakket med biler, og det er ingen mulighet for bevegelse i trafikken. Derfor er nødvendigvis hastigheten $v(u_{max}) = 0$.

Å definere funksjonen slik reflekterer fysisk realitet. Hadde ikke $v(u_{max}) = 0$ blitt definert eksplisitt, ville vi fått et udefinert uttrykk, dvs. logaritmen til 0, i hoveduttrykket for funksjonen.

Plott: I fig. 1 ser vi grafen til den logaritmiske funksjonen. Plottet bruker samme eksempelverdier for $v_{max} = 60$ km/t og $u_{max} = 100$ biler/km, for å illustrere et realistisk scenario.



Figur 1: Oppgave 1b-iii: Plott av en logaritmisk funksjon.

Python-kode for å lage grafen til den logaritmiske funksjonen:

```
# Plott av den logaritmiske funksjonen
import numpy as np
import matplotlib.pyplot as plt

def v_log(u, v_max, u_max):
    """
    Logaritmisk hastighetsfunksjon.

    Args:
        u: Tetthet (array eller enkeltverdi).
        v_max: Maksimal hastighet.
        u_max: Maksimal tetthet.

    Returns:
        Hastighet (array eller enkeltverdi).
    """
    if isinstance(u, (int, float)):
        if u >= u_max:
            return 0
        else:
            return v_max * np.log((u_max + 1) / (u + 1)) / np.log(u_max + 1)
    else:
        v = np.zeros_like(u, dtype=float)
        mask = u < u_max
        v[mask] = v_max * np.log((u_max + 1) / (u[mask] + 1)) / np.log(u_max + 1)
```

```

    return v

# Definerer parametre
v_max = 60 # km/t
u_max = 100 # biler/km

# Lager et array med tetthetsverdier
u_values = np.linspace(0, u_max, 400)

# Beregner hastighetene
v_values = v_log(u_values, v_max, u_max)

# Tegner grafen
plt.figure(figsize=(8, 6))
plt.plot(u_values, v_values)
plt.xlabel("Tetthet (u) [biler/km]")
plt.ylabel("Hastighet (v) [km/t]")
plt.title("Logaritmisk hastighetsfunksjon")
plt.grid(True)
plt.xlim(0, u_max)
plt.ylim(0, v_max * 1.1)
plt.show()

```

Consinus-funksjon

$$v(u) = v_{max} \cdot \cos\left(\frac{\pi}{2} \cdot \frac{u}{u_{max}}\right)$$

1. Antagelse: $v \geq 0$:

For $0 \leq u \leq u_{max}$ er $0 < \frac{u}{u_{max}} \leq 1$.

Dermed er $0 \leq \frac{\pi}{2} \cdot \frac{u}{u_{max}} \leq \frac{\pi}{2}$

Cosinus-funksjonen er ikke-negativ i intervallet $[0, \frac{\pi}{2}]$.

siden v_{max} er positiv, er $v(u) = v_{max} \cdot \cos\left(\frac{\pi}{2} \cdot \frac{u}{u_{max}}\right) \geq 0$

2. Antagelse: $v \leq v_{max}$, hvor v_{max} er maksfart:

Cosinus-funksjonen har verdier mellom -1 og 1 . I intervallet fra forrige punkt, $[0, \frac{\pi}{2}]$, er cosinus-funksjonen mellom 0 og 1 .

Dermed er $\cos\left(\frac{\pi}{2} \cdot \frac{u}{u_{max}}\right) \leq 1$

Siden v_{max} er positiv, er $v(u) = v_{max} \cdot \cos\left(\frac{\pi}{2} \cdot \frac{u}{u_{max}}\right) \leq v_{max}$

3. Antagelse: $\frac{dv}{du} \leq 0$: Vi deriverer $v(u)$ med hensyn på u :

$$\begin{aligned}
\frac{dv}{du} &= v_{max} \cdot \frac{d}{du} \left(\cos \left(\frac{\pi}{2} \cdot \left(\frac{u}{u_{max}} \right) \right) \right) \\
&= v_{max} \cdot \left(-\sin \left(\frac{\pi}{2} \cdot \left(\frac{u}{u_{max}} \right) \right) \right) \cdot \left(\frac{\pi}{2} \right) \cdot \left(\frac{1}{u_{max}} \right) \\
&= - \left(v_{max} \cdot \frac{\pi}{2 \cdot u_{max}} \right) \cdot \sin \left(\left(\frac{\pi}{2} \right) \cdot \left(\frac{u}{u_{max}} \right) \right)
\end{aligned}$$

For $0 \leq u \leq u_{max}$ er $0 \leq \left(\frac{\pi}{2} \cdot \frac{u}{u_{max}} \right) \leq \frac{\pi}{2}$

Vi vet at cos derivert er $-\sin$. Sinus-funksjonen er ikke-negativ på intervallet $[0, \frac{\pi}{2}]$.

v_{max} , π , og u_{max} er positive konstanter.

Dermed er:

$$-\frac{v_{max} \cdot \pi}{2 \cdot u_{max}} \cdot \sin \left(\frac{\pi}{2} \cdot \frac{u}{u_{max}} \right) \leq 0$$

Altså $\frac{dv}{du} \leq 0$.

4. Antagelse: $v(0) = v_{max}$ når tettheten $u = 0$:

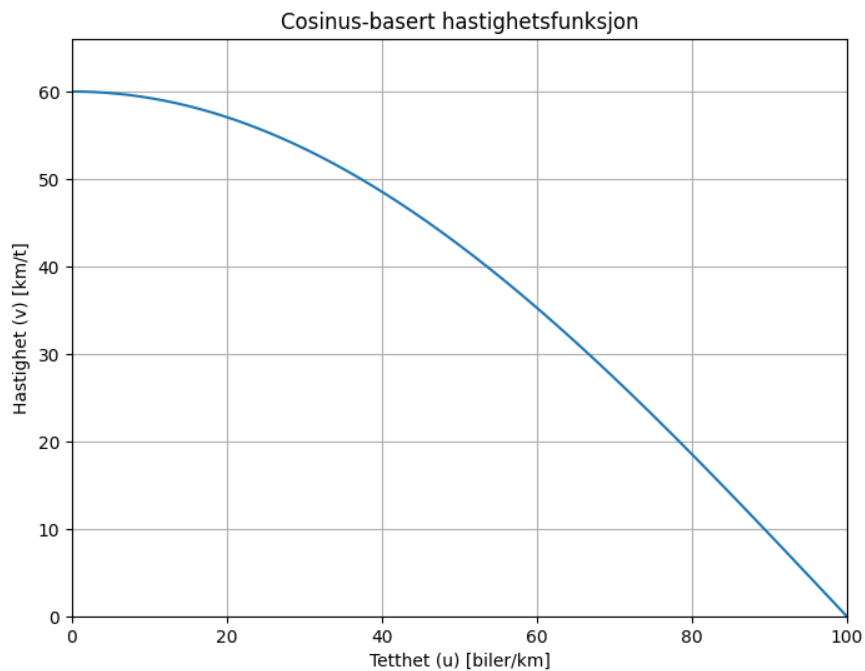
Setter inn $u = 0$ i uttrykket

$$\begin{aligned}
v(0) &= v_{max} \cdot \cos \left(\frac{\pi}{2} \cdot \frac{0}{u_{max}} \right) \\
&= v_{max} \cdot \cos(0) \\
&= v_{max} \cdot 1 \\
&= v_{max}
\end{aligned}$$

5. Antagelse: $v(u_{max}) = 0$ når tettheten $u = u_{max}$ er størst mulig:

$$\begin{aligned}
v(u_{max}) &= v_{max} \cos \left(\frac{\pi}{2} \cdot \frac{u_{max}}{u_{max}} \right) \\
&= v_{max} \cdot \cos \left(\frac{\pi}{2} \right) \\
&= v_{max} \cdot 0 = 0
\end{aligned}$$

Plott: I fig. 2 ser vi grafen til cosinus-funksjonen. Plottet bruker samme eksempelverdier for $v_{max} = 60$ km/t og $u_{max} = 100$ biler/km, for å illustrere et realistisk scenario.



Figur 2: Oppgave 1b-iii: Plott av cosinus-funksjonen.

Python-kode for å lage grafen til cosinus-funksjonen:

```
# Plott av cosinus-funksjonen
import numpy as np
import matplotlib.pyplot as plt

def v_cos(u, v_max, u_max):
    """
    Cosinus-basert hastighetsfunksjon.

    Args:
        u: Tetthet (array eller enkeltverdi).
        v_max: Maksimal hastighet.
        u_max: Maksimal tetthet.

    Returns:
        Hastighet (array eller enkeltverdi).
    """
    return v_max * np.cos((np.pi / 2) * (u / u_max))

# Definerer parametre
v_max = 60 # km/t
u_max = 100 # biler/km

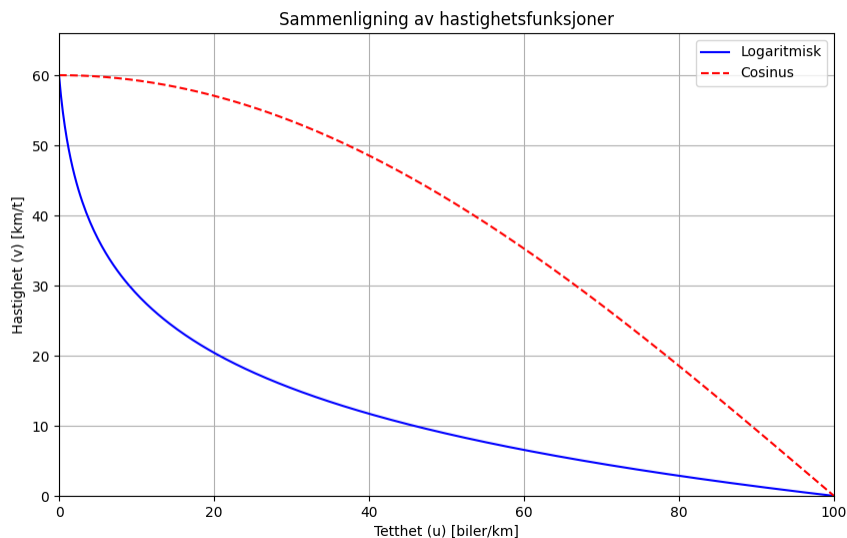
# Lager et array med tetthetsverdier
u_values = np.linspace(0, u_max, 400)
```

```
# Beregner hastighetene
v_values = v_cos(u_values, v_max, u_max)

# Tegner grafen
plt.figure(figsize=(8, 6))
plt.plot(u_values, v_values)
plt.xlabel("Tetthet (u) [biler/km]")
plt.ylabel("Hastighet (v) [km/t]")
plt.title("Cosinus-basert hastighetsfunksjon")
plt.grid(True)
plt.xlim(0, u_max)
plt.ylim(0, v_max * 1.1)
plt.show()
```

Samleplott

I fig. 3 ser vi grafen til begge funksjonene i samme plott. Selv om de har ulik bane, avtar hastigheten v mot null ved økende tetthet u , i henhold til antagelsene om sammenhengen mellom biltetthet og hastighet. Begge plottene er med samme eksempelverdier for $v_{max} = 60$ km/t og $u_{max} = 100$ biler/km.



Figur 3: Oppgave 1b-iii: Logaritmisk og cosinus-funksjon i samme plott.

2.1.3 c) Den deriverte

i)

Hva er måleenhetene til $\frac{dv}{du}$, $\frac{\partial u}{\partial t}$, $\frac{\partial u}{\partial x}$, $\frac{\partial v}{\partial t}$ og $\frac{\partial v}{\partial x}$?

Måleenhet 1: $\frac{dv}{du}$

Endring i hastighet ved endringer i biltetthet.

- v : Hastighet. Måles i km/t eller m/s. Vi bruker SI-enheten, m/s.

- u : Tetthet. Definert til biler per meter, biler/m.
- $\frac{dv}{du}$: Endring i hastighet, per endring i tetthet.

Utledning:

$$\begin{aligned}\frac{\text{Endring hastighet}}{\text{Endring biltetthet}} &= \frac{\text{m/s}}{\text{biler/m}} \\ &= \frac{\text{m}}{\text{s}} \cdot \frac{\text{m}}{\text{biler}} \\ &= \frac{\text{m}^2}{\text{s} \cdot \text{biler}}\end{aligned}$$

Forklaring: Se oppgave 01-C-ii, Måleenhet 01 (kapittel 2.1.3).

Måleenhet 2: $\frac{\partial u}{\partial t}$

Endring i biltetthet over tid, ved et gitt punkt

- u : Tetthet, måles i $\frac{\text{biler}}{\text{meter}}$.
- t : Tid, måles i sekunder (s).
- $\frac{\partial u}{\partial t}$: Endring i tetthet per endring i tid, hvor posisjon x holdes konstant.

Utledning:

$$\begin{aligned}\frac{\text{Endring biltetthet}}{\text{Endring tid}} &= \frac{\text{biler/m}}{\text{sekund}} \\ &= \frac{\text{biler}}{\text{m} \cdot \text{s}}\end{aligned}$$

Forklaring: Se oppgave 01-C-ii, Måleenhet 02 (kapittel 2.1.3).

Måleenhet 3: $\frac{\partial u}{\partial x}$

Endring i biltetthet langs veistrekningen ved posisjon x , ved et gitt tidspunkt.

- u : Tetthet, måles i $\frac{\text{biler}}{\text{meter}}$.
- x : Posisjon, måles i meter (m)
- $\frac{\partial u}{\partial x}$: Endring i tetthet per endring i posisjon, hvor tiden t holdes konstant.

Utledning:

$$\begin{aligned}\frac{\text{Endring biltetthet}}{\text{Endring posisjon}} &= \frac{\text{biler/m}}{\text{m}} \\ &= \frac{\text{biler}}{\text{m}^2}\end{aligned}$$

Forklaring: Se oppgave 01-C-ii, Måleenhet 03 (kapittel 2.1.3).

Måleenhet 4: $\frac{\partial v}{\partial t}$

Endring i hastighet per endring i tid, ved et gitt punkt.

- v : Hastighet, måles i $\frac{\text{meter}}{\text{sekund}}$.
- t : Tid, måles i sekund (s)
- $\frac{\partial v}{\partial t}$: Endring i hastighet per endring i tid, hvor posisjon x holdes konstant.

Utleddning:

$$\frac{\text{Endring hastighet}}{\text{Endring tid}} = \frac{\text{m/s}}{s} = \frac{\text{m}}{s^2}$$

Forklaring: Se oppgave 01-C-ii, Måleenhet 04 (kapittel 2.1.3).

Måleenhet 5: $\frac{\partial v}{\partial x}$

Endring i hastigheten med posisjon langs veien, ved et gidd tidspunkt.

- v : Hastighet, måles i $\frac{\text{meter}}{\text{sekund}}$.
- x : Posisjon, målet i meter.
- $\frac{\partial v}{\partial x}$: Endring i hastighet per endring i posisjon, ved et gitt tidspunkt.

Utleddning:

$$\frac{\text{Endring hastighet}}{\text{Endring posisjon}} = \frac{\text{m/s}}{\text{m}} = \frac{\text{m}}{s \cdot \text{m}} = \frac{1}{s}$$

Forklaring: Se oppgave 01-C-ii, Måleenhet 05 (kapittel 2.1.3).

Oppsummert måleenheter

I tabell 1 ser vi en sammenstilling av enhetene.

Derivert	Måleenhet	Forklaring
$\frac{dv}{du}$	$(\text{m}^2/(\text{s} \cdot \text{biler}))$	Endring i hastighet per endring i tetthet.
$\frac{\partial u}{\partial t}$	$(\text{biler}/(\text{m} \cdot \text{s}))$	Endring i tetthet per tid, ved et fast punkt.
$\frac{\partial u}{\partial x}$	$(\text{biler}/\text{m}^2)$	Endring i tetthet per lengde, ved et fast tidspunkt.
$\frac{\partial v}{\partial t}$	(m/s^2)	Endring i hastighet per tid, ved et fast punkt.
$\frac{\partial v}{\partial x}$	$(1/\text{s})$	Endring i hastighet per lengde, ved et fast tidspunkt.

Tabell 1: Oppgave 1c: Oversikt over deriverte, måleenheter og forklaringer.

ii)

Forklar hva hver av disse deriverte forteller oss om trafikkflyten.

Positiv x-retning settes lik kjøreretning.

$\frac{dv}{du}$: **Hvordan hastighet endres med biltetthet**

Som vist i oppgave 01-c-i, Måleenhet 01 (kapittel 2.1.3), er enheten:

$$\frac{\text{m}^2}{\text{s} \cdot \text{biler}}$$

Måleenheten beskriver hvor mye hastigheten til bilene endrer seg, i meter per sekund, når tettheten øker med én bil per meter. Forteller hvor følsom hastigheten v er for endringer i biltetthet u .

- **Negative verdier:** Indikerer at hastigheten synker når tettheten øker. Jo flere biler, jo saktere går det. Størrelsen på den negative verdien forteller oss hvor følsom hastigheten er for endringer i tetthet. En stor negativ verdi betyr at hastigheten synker raskt med økende tetthet (f.eks. i en situasjon der kø lett dannes). En liten negativ verdi betyr at hastigheten ikke endrer seg så mye selv om tettheten øker (f.eks. på en vei med god kapasitet).
- **Null:** Vi har tidligere vist at ved $u = 0$, er $v(0) = v_{max}$. Dette betyr at når tettheten går mot null, er hastigheten lik maksimal hastighet (f.eks. fartsgrensen). Dette gir mening, fordi når det er svært få biler på veien kan de holde f.eks. maksimal tillatte hastighet.

$\frac{\partial u}{\partial t}$: **Hvordan biltetthet endres over tid**

Som vist i oppgave 01-c-i, Måleenhet 02 (kapittel 2.1.3), er enheten:

$$\frac{\text{biler}}{\text{m} \cdot \text{s}}$$

Måleenheten beskriver endring i biltetthet over tid, ved et fast punkt. Altså hvordan tettheten endrer seg *lokalt*, på et bestemt punkt, over tid. Dette viser opphopning eller uttømming av biler på en gitt veistrekke, f.eks. ved kødannelse eller oppløsning av kø.

Beskriver hvordan tettheten av biler på et bestemt punkt på veien utvikler seg over tid.

Ved positive verdier, $\frac{\partial u}{\partial t} > 0$, betyr det at flere biler samler seg i dette punktet, hvilket kan indikere en kødannelse.

Ved negative verdier, $\frac{\partial u}{\partial t} < 0$, betyr det at bilene forsvinner fra punktet, f.eks. ved at de akselererer, at veiene blir mer åpne eller at en kø løses opp.

$\frac{\partial u}{\partial x}$: **Hvordan biltettheten varierer langs veien**

Som vist i oppgave 01-c-i, Måleenhet 03 (kapittel 2.1.3), er enheten:

$$\frac{\text{biler}}{\text{m}^2}$$

Måleenheten beskriver hvordan biltetthet (i biler per meter) endrer seg per meter langs veien, ved et gitt tidspunkt t .

- **Positive verdier:** $\frac{\partial u}{\partial x} > 0$, betyr det at biltettheten øker fremover langs veien (positiv x-retning), hvilket kan indikere at det bygges opp kø foran.
- **Negative verdier:** $\frac{\partial u}{\partial x} < 0$, betyr det at tettheten avtar foran, f.eks at en kø løser seg opp.
- **Null:** Tettheten er konstant langs veien ved det gitte tidspunktet.
- **Eksempel:** På en vei med innsnevring vil u øke gradvis mot innsnevringen, noe som gir positiv $\frac{\partial u}{\partial x}$. Lenger frem, litt etter innsnevringen vil trafikken finne en ny flyt og u minker, noe som gir negativ $\frac{\partial u}{\partial x}$.

$\frac{\partial v}{\partial t}$: Hvordan trafikkhastigheten i et punkt endres over tid

Som vist i oppgave 01-c-i, Måleenhet 04 (kapittel 2.1.3), er enheten:

$$\frac{\text{m}}{\text{s}^2}$$

Måleenheten forteller oss hvor mye hastigheten (i meter per sekund) til bilene, ved et gitt punkt på veien, endrer seg per sekund. Dette er ikke akselerasjonen til enkeltbiler, men hvordan hastigheten i snitt endres ved et fast punkt.

- **Positiv verdier:** $\frac{\partial v}{\partial t} > 0$, betyr at hastigheten øker, f.eks ved at en kø løser seg opp, og bilene som passerer punktet øker hastigheten sin.
- **Negative verdier:** $\frac{\partial v}{\partial t} < 0$, betyr at hastigheten synker, f.eks ved at en kø begynner å dannes og bilene som passerer punktet bremses ned.
- **Null:** Hastigheten ved punktet er konstant.
- **Eksempel:** Hvis trafikken plutselig stopper opp på grunn av en ulykke lenger fremme, vil v synke raskt og gir negativ $\frac{\partial v}{\partial t}$. Når trafikken kan fortsette, vil bilene som helhet akselerere, og v vil stige og gir positiv $\frac{\partial v}{\partial t}$.

$\frac{\partial v}{\partial x}$: Hvordan trafikkhastigheten varierer langs veien

Som vist i oppgave 01-c-i, Måleenhet 05 (kapittel 2.1.3), er enheten:

$$\frac{1}{\text{s}}$$

Måleenheten beskriver hvor mye hastigheten til bilene endrer seg per meter langs veien, ved et gitt tidspunkt, når vi beveger oss i positiv x-retning.

- **Positive verdier:** $\frac{\partial v}{\partial x} > 0$, betyr det at hastigheten øker fremover langs veien, f.eks ved at en kø løsner opp.
- **Negative verdier:** $\frac{\partial v}{\partial x} < 0$, vil hastigheten avta fremover, f.eks ved at en kø dannes.
- **Null:** Hastigheten er konstant langs veien ved det gitte tidspunktet.
- En høy verdi betyr at hastigheten varierer mye over korte avstander, f.eks. i områder med skiftende fartsgrenser eller køer, mens en lav verdi betyr at hastigheten er mer stabil langs veien.
- **Eksempel:** Ved en endring i fartsgrense fra 80 km/t til 50 km/t, vil v synke ved dette punktet, noe som gir negativ $\frac{\partial v}{\partial x}$. Og omvendt, ved en økning i fartsgrense, vil v øke i punktet, og gi positiv $\frac{\partial v}{\partial x}$.

Oppsummert

Disse deriverte gir oss et detlert bilde av hvordan trafikkflyten endrer seg med både tid og posisjon. Ved å kombinere informasjon fra alle disse, kan det dannes en god forståelse for trafikkdynamikken på en veistrekning. For eksempel, hvis $\frac{\partial u}{\partial t}$ og $\frac{\partial v}{\partial t}$ er negativ ved et punkt, kan det tyde på at en kø er i ferd med å bygges opp ved dette punktet.

I tabellen under (table 2) ser vi en sammenstilling av de forskjellige trafikkderivate", deres praktiske betydning, og hva positive og negative verdier betyr.

Derivert	Fysisk betydning	Positiv verdi betyr	Negativ verdi betyr
$\frac{dv}{du}$	Endring i hastighet med biltetthet	Høyere hastighet med økt tetthet (ikke realistisk)	Lavere hastighet med økt tetthet (realistisk)
$\frac{\partial u}{\partial t}$	Endring i biltetthet over tid	Kø bygger seg opp	Kø løser seg opp
$\frac{\partial u}{\partial x}$	Endring i biltetthet langs veien	Økt tetthet fremover (inn i kø)	Lavere tetthet fremover (ut av kø)
$\frac{\partial v}{\partial t}$	Endring i hastighet i et punkt over tid	Trafikken akselererer	Trafikken bremser ned
$\frac{\partial v}{\partial x}$	Endring i hastighet langs veien	Trafikken går raskere fremover	Trafikken går saktere fremover

Tabell 2: Oppgave 1c: Oppsummering av hvordan de deriverte påvirker trafikken.

2.1.4 d) Utrekning av fluksen

i)

Forklar hvorfor fluksen er gitt ved

$$J(x, t) = J(u(x, t)) = u(x, t)v(u(x, t))$$

der $v(u)$ er hastigheten til bilene når biltettheten er u . Det vil si at fluksen $J = J(u)$ også er en funksjon av tettheten u .

Fluksen $J(x, t)$ beskriver hvor mange biler som passerer et gitt punkt x på veien per tidsenhet (sekund), ved tidspunkt t . Fra oppgaven får vi opplyst at " J har måleenhet antall biler per sekund". Dette kan anses som en grunnleggende størrelse i trafikkanalysen, og kan betegnes som *gjennomstrømning*.

Utleddning for hvorfor fluks er gitt ved produktet v tetthet og hastighet:

Anta et lite tidsintervall, Δt , sett på ved et fst punkt x langs veien.

Gjennomsnittlig tetthet:

I løpet av dette tidsintervallet Δt vil bilene ved posisjon x ha en viss gjennomsnittlig hastighet, som vi kan representere med $u(x, t)$, altså tetthet, om måles i biler per meter (biler/m).

Gjennomsnittlig hastighet:

Bilene ved posisjon x vil ha en gjennomsnittlig hastighet i løpet av Δt . Siden vi antar t hastigheten er en funksjon av tettheten, kan vi skrive denne gjennomsnittlige hastigheten som $v(u(x, t))$. Hastigheten måles i meter per sekund (m/s).

Tilbakelagt distanse:

I løpet av tidsintervallet Δt vil bilene (i gjennomsnitt) bevege seg en distanse gitt ved:

$$\text{distanse} = \text{hastighet} \cdot \text{tid} = v(u(x, t)) \cdot \Delta t$$

Dette er distansen bilene ved posisjon x har beveget seg i løpet av tidsintervallet.

Antall biler som passerer: Alle bilene som befant seg innenfor denne distansen, *bak* punktet x , ved starten av tidsintervallet, vil ha passert punktet x i løpet av Δt . Antallet bil er som passerer x i løpet av Δt er derfor gitt ved:

$$\text{antall biler} = \text{tetthet} \cdot \text{distanse} = u(x, t) \cdot v(u(x, t)) \cdot \Delta t$$

Fluks:

Fluks er definert som *antall biler per tidsenhet*. Vi deler derfor antall biler som passerer, på Δt :

$$\begin{aligned} J(x, t) &= \frac{\text{antall biler}}{\Delta t} \\ &= \frac{u(x, t) \cdot v(u(x, t)) \cdot \Delta t}{\Delta t} \\ &= u(x, t) \cdot v(u(x, t)) \end{aligned}$$

Enheten blir da:

$$\frac{\text{biler}}{\text{m}} \cdot \frac{\text{m}}{\text{s}} = \frac{\text{biler}}{\text{s}}$$

Dette stemmer med definisjonen av fluks gitt i oppgaven.

Vi kan tenke oss en veistrekning:

- **Lav tetthet (liten u):** Få biler på veien. De kan kjøre fort (høy v).
- **Høy tetthet (stor u):** Mange biler på veien. De må kjøre sakte (lav v).

Dette viser at fluksen J er direkte avhengig av både tetthet og hastighet. Hvis enten tettheten eller hastigheten øker, vil flere biler passere per sekund, men hvis én av dem blir for lav, reduseres fluksen. Dette forklarer hvorfor fluksen ofte når et maksimum for en viss tetthet, før den synker igjen når veien blir for tettpakket.

2.1.5 e) Maksimal fluks

Fluksen er gitt ved:

$$J(u) = u \cdot v(u)$$

i)

Hvis hastighetsfunksjonen er $v(u) = v_{max} \left(1 - \frac{u}{u_{max}}\right)$, hva er hastigheten når strømmen (fluksen) er maksimal? Hva er den maksimale fluksen?

Vi setter inn den gitte hastighetsfunksjonen i formelen for fluks:

$$\begin{aligned} J(u) &= u \cdot v_{max} \left(1 - \frac{u}{u_{max}}\right) \\ &= v_{max} \left(u - \frac{u^2}{u_{max}}\right) \end{aligned}$$

Finner biltettheten som gir maksimal fluks

For å finne biltettheten u som gir maksimal fluks, deriverer vi $J(u)$ med hensyn på u og setter den lik null:

$$\begin{aligned} \frac{dJ}{du} &= \frac{d}{du} \left[v_{max} \cdot \left(u - \frac{u^2}{u_{max}}\right) \right] \\ &= v_{max} \cdot \left(1 - \frac{2u}{u_{max}}\right) \end{aligned}$$

Setter $\frac{dJ}{du} = 0$:

$$v_{max} \cdot \left(1 - \frac{2u}{u_{max}}\right) = 0$$

Siden $v_{max} \neq 0$ kan vi dele på begge sider av likningen på v_{max} :

$$\begin{aligned} \frac{v_{max} \cdot \left(1 - \frac{2u}{u_{max}}\right)}{v_{max}} &= \frac{0}{v_{max}} \\ 1 - \frac{2u}{u_{max}} &= 0 \\ \frac{2u}{u_{max}} &= 1 \\ u &= \frac{u_{max}}{2} \end{aligned}$$

Vi finner altså at fluksen er maksimal ved biltetthet $u = \frac{u_{max}}{2}$

Vi utfører andrederiverttesten for å forsikre at dette er et maksimum, ikke et sadelpunkt eller minimum.

Vi deriverer $\frac{dJ}{du}$ en gang til:

$$\begin{aligned} \frac{d^2J}{du^2} &= \frac{d}{du} \left[v_{max} \cdot \left(1 - \frac{2u}{u_{max}}\right) \right] \\ &= -\frac{2v_{max}}{u_{max}} \end{aligned}$$

Siden v_{max} og u_{max} er positive konstanter, er $\frac{d^2J}{du^2}$ negativ. En negativ andrederivert betyr at vi har et maksimumspunkt.

Hastighet ved maksimal fluks

Vi setter $u = \frac{u_{max}}{2}$ inn i hastighetsfunksjonen:

$$\begin{aligned}v\left(\frac{u_{max}}{2}\right) &= v_{max} \cdot \left(1 - \frac{\frac{u_{max}}{2}}{u_{max}}\right) \\&= v_{max} \cdot \left(1 - \frac{1}{2}\right) \\&= v_{max} \cdot \frac{1}{2} \\&= \frac{v_{max}}{2}\end{aligned}$$

Maksimal fluks

Vi har funnet at maksimal fluks oppstår når $u = \frac{u_{max}}{2}$. Dermed finner vi maksimal fluks, J_{max} , ved å sette inn $u = \frac{u_{max}}{2}$ i uttrykket for $J(u)$:

$$\begin{aligned}J_{max} &= J\left(\frac{u_{max}}{2}\right) \\&= v_{max} \cdot \left(\frac{u_{max}}{2} - \frac{(u_{max}/2)^2}{u_{max}}\right) \\&= v_{max} \cdot \left(\frac{u_{max}}{2} - \frac{u_{max}^2/4}{u_{max}}\right) \\&= v_{max} \cdot \left(\frac{u_{max}}{2} - \frac{u_{max}}{4}\right) \\&= v_{max} \cdot \frac{u_{max}}{4} \\&= \frac{v_{max} \cdot u_{max}}{4}\end{aligned}$$

Oppsummert i)

For å oppsummere:

- Fluksen er maksimal ved biltetthet: $u = \frac{u_{max}}{2}$
- Hastighet ved maksimal fluks: $v = \frac{v_{max}}{2}$
- Maksimal fluks: $J_{max} = \frac{u_{max} \cdot v_{max}}{4}$

ii)

Regn ut hastigheten ved maksimal fluks for hastighetsfunksjonen $v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}}\right)^p\right)$.

Definerer fluksen $J(u)$:

Vi begynner med å definere fluksen som produktet av tetthet og hastighet:

$$J(u) = u \cdot v(u)$$

Setter vi hastighetsfunksjonen inn i fluksfunksjonen får vi:

$$\begin{aligned} J(u) &= u \cdot v_{max} \left(1 - \left(\frac{u}{u_{max}} \right)^p \right) \\ &= v_{max} \left(u - u \left(\frac{u}{u_{max}} \right)^p \right) \\ &= v_{max} \left(u - \frac{u^{p+1}}{u_{max}^p} \right) \end{aligned}$$

Finner biltettheten som gir maksimal fluks:

For å finne verdien av u som gir maksimal fluks, deriverer vi $J(u)$ med hensyn på u og setter den lik null:

$$\begin{aligned} \frac{dJ}{du} &= \frac{d}{du} \left[v_{max} \left(1 - (p+1) \frac{u^p}{u_{max}^p} \right) \right] \\ &= v_{max} \left(1 - \frac{(p+1)u^p}{u_{max}^p} \right) \end{aligned}$$

Setter $\frac{dJ}{du} = 0$:

$$v_{max} \left(1 - \frac{(p+1)u^p}{u_{max}^p} \right) = 0$$

Siden $v_{max} \neq 0$ (maksimalhastigheten er positiv), kan vi dele begge sider på v_{max} og løser for u :

$$\begin{aligned} 1 - (p+1) \frac{u^p}{u_{max}^p} &= 0 \\ (p+1) \frac{u^p}{u_{max}^p} &= 1 \\ u^p &= \frac{u_{max}^p}{p+1} \\ u &= u_{max} \left(\frac{1}{p+1} \right)^{\frac{1}{p}} = (p+1)^{-\frac{1}{p}} u_{max} \end{aligned}$$

Dette gir den biltettheten som maksimerer fluksen.

Andrederiverttest

For å bekrefte at det er et maksimumspunkt, utfører vi andrederiverttesten, altså deriverer $\frac{dJ}{du}$ en gang til:

$$\begin{aligned} \frac{d^2J}{du^2} &= \frac{d}{du} \left[v_{max} \cdot \left(1 - \frac{(p+1)u^p}{u_{max}^p} \right) \right] \\ &= -v_{max} \cdot \frac{p(p+1)u^{p-1}}{u_{max}^p} \end{aligned}$$

Setter vi inn $u = (p+1)^{-\frac{1}{p}}$ får vi:

$$\begin{aligned}\frac{d^2 J}{du^2} &= -\frac{v_{max} \cdot p \cdot (p+1)}{u_{max}^p} \cdot \left((p+1)^{-\frac{1}{p}} u_{max} \right)^{p-1} \\ &= -v_{max} \cdot p \cdot (p+1)^{\frac{1}{p}} \cdot u_{max}^{-1}\end{aligned}$$

Siden $p > 0$ (fra oppgaveteksten), $v_{max} > 0$, og $u_{max} > 0$, er $\frac{d^2 J}{du^2}$ negativ. En negativ andrederivert betyr at vi har funnet et maks punkt.

Hastighet ved maksimal fluks

Setter inn $u = (p+1)^{-\frac{1}{p}}$ i uttrykket for $v(u)$:

$$\begin{aligned}v \left((p+1)^{-\frac{1}{p}} u_{max} \right) &= v_{max} \left(1 - \left(\frac{(p+1)^{-\frac{1}{p}} u_{max}}{u_{max}} \right)^p \right) \\ &= v_{max} \left(1 - \left((p+1)^{-\frac{1}{p}} \right)^p \right) \\ &= v_{max} (1 - (p+1)^{-1}) \\ &= v_{max} \left(\frac{p+1-1}{p+1} \right) \\ &= v_{max} \left(\frac{p}{p+1} \right)\end{aligned}$$

Dette gir hastigheten ved maksimal fluks.

Oppsummert ii)

For å oppsummere:

- Tetthet ved maksimal fluks: $u = (p+1)^{-\frac{1}{p}} \cdot u_{max}$
- Hastighet ved maksimal fluks: $v = v_{max} \left(\frac{p}{p+1} \right)$

Vi ser at altså at hastigheten ved maksimal fluks avhenger av eksponenten p i funksjonen. Hastigheten ved maksimal fluks for hastighetsfunksjonen er altså:

$$v = v_{max} \left(\frac{p}{p+1} \right)$$

2.1.6 f) Differensiallikningen

Sjekk ved regning at begge sider av likningene har samme måleenhet. Hva er måleenheten?

$$\frac{\partial u(x, t)}{\partial t} = -\frac{\partial J(x, t)}{\partial x} \quad (11)$$

Vi begynner med å sjekke hver av sidene for seg.

Venstre side (VS): $\frac{\partial u(x,t)}{\partial t}$ representerer endring i tetthet u over tid t .

Tetthet $u(x,t)$ har enhet $\frac{\text{biler}}{\text{m}}$ (antall biler per meter).

Tid, t , har enhet s (sekunder).

Derfor har $\frac{\partial u(x,t)}{\partial t}$ enheten $\frac{\text{biler/m}}{\text{s}} = \frac{\text{biler}}{\text{m}\cdot\text{s}}$ (biler per meter per sekund).

Høyre side (HS): $-\frac{\partial J(x,t)}{\partial x}$ representerer den negative endringen u fluks (J) langs posisjon (x).

Fluks, $J(x,t)$, har enheten $\frac{\text{biler}}{\text{s}}$ (antall biler som passerer per sekund).

Posisjon, x , har enhet m (meter).

Derfor har $-\frac{\partial J(x,t)}{\partial x}$ enheten: $\frac{\text{biler/s}}{\text{m}} = \frac{\text{biler}}{\text{m}\cdot\text{s}}$ (biler per meter per sekund).

Konklusjon: Begge sider av likningen har samme måleenhet: $\frac{\text{biler}}{\text{m}\cdot\text{s}}$

Denne enheten viser endring i biltetthet over tid. Det forteller hvor mye tettheten av biler endrer seg per sekund på et gitt sted.

—

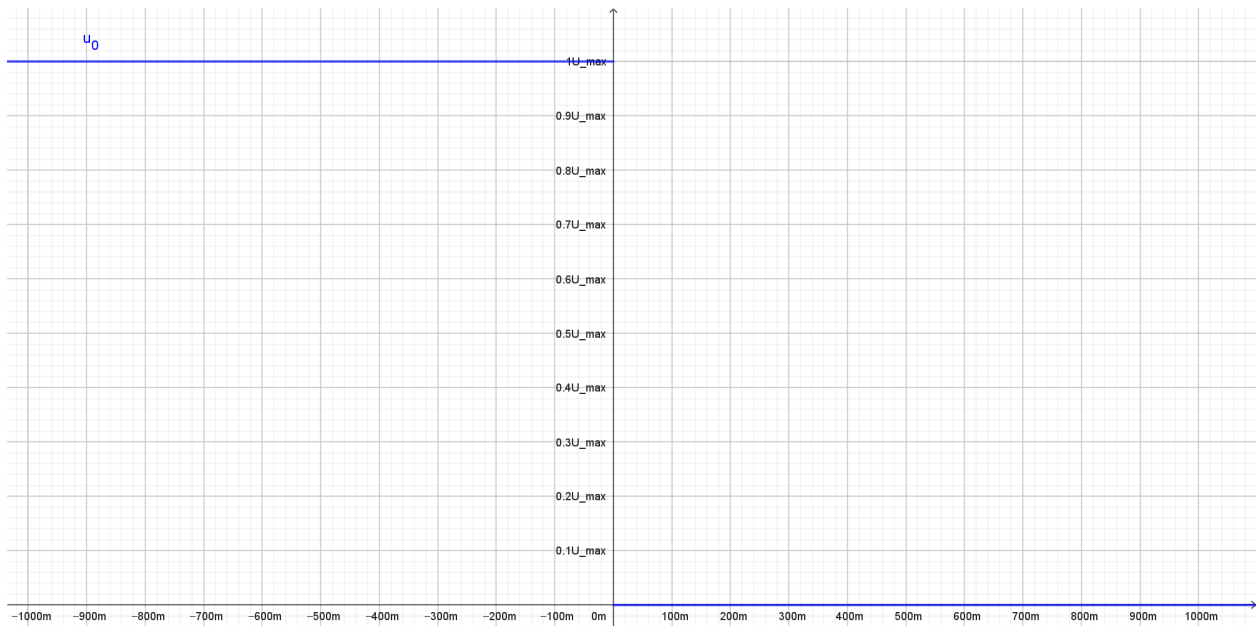
2.2 Oppgave 2

2.2.1 a) Initialverdibetingelsen

Vi vet at for $x < 0$ ved $t = 0$ står bilene stille og har maksimal tetthet u_{max} , og at for $x > 0$ ved $t = 0$ er det ingen biler, dvs. $u(x, 0) = 0$ som gir oss funksjonen

$$u_0(x) = \begin{cases} u_{\max}, & x < 0 \\ 0, & x \geq 0 \end{cases} \quad (1)$$

Som gir oss skissen:



Figur 4: Oppgave 2a: Skisse for en graf for de gitte initialverdiene

Skissen ovenfor forteller oss at det er maksimal tetthet for biler som venter på grønt og ingen biler forbi lyskrysset når $t = 0$.

2.2.2 b) Verdiene v_{max} og u_{max} og randbetingelsen for $x = 1000$.

I Norge er det vanlig med en fartsgrense mellom 60km/t og 100km/t, derfor kan vi rimelig anta at maksimal hastighet på 100km/t, som kan rundes opp til 28m/s.

Gjennomsnittslengden på en bil er 495cm Merg (2023). Om man antar at det er ca. en bils mellomrom mellom hver bil ved maksimal tetthet, og runder opp billengden til 5m, kan man si at det er 1 bil hver 10m.

Da er verdiene v_{max} og u_{max} bestemt slik:

- $v_{max} = 28m/s$
- $u_{max} = 0.1biler/m$

Om man antar at randbetingelsen er $u(1000, t) = 0$ vil det bety at det ikke er biler etter posisjonen $x = 1000$ uansett hvor lang tid det har gått. Det kan tolkes som om at veien ender ved $x = 1000$ eller forbi $x = 1000$ er utenfor modellens rekkevidde. Om $u(1000, t) = u_{max}$ kan det bety en trafikkork ved $x = 1000$

2.2.3 c) Løs differensiallikningen

Skal løse og animere

$$u_t + J(u)_x = 0, \quad J(u) = u \cdot v(u) \quad (2)$$

ved bruk av Lax-Friedrichs metode med hensyn på tiden t for tre ulike p -verdier, $p = 1, 2, 5$. Er gitt at hastighet v avhenger av tetthet u og varierer med p ,

$$v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}} \right)^p \right) \quad (3)$$

randbetingelsen $u(1000, t) = 0$, og Lax-Friedrichs metode Wikipedia (b)

$$u_j^{n+1} = \frac{\Delta t}{2}(u_{j+1}^n + u_{j-1}^n) - \frac{\Delta t}{2\Delta x}(J(u_{j+1}^n) - J(u_{j-1}^n)) \quad (4)$$

Der u_i^n er verdien til $u(x, t)$ i det diskrete rutenettet. Første steg er å diskretisere ligningen. Tidssteg må velges for å oppfylle CFL-betingelsen for eksplisitte metoder Wikipedia (a) :

$$\frac{\Delta t}{\Delta x} \cdot v_{max} \leq 1 \quad (5)$$

Velger da vilkårlig N_x for å få en verdi for Δx

$$\Delta x = \frac{\text{banelengde}}{N_x} \quad (6)$$

$$\Delta x = \frac{2000m}{2000} \quad (7)$$

$$\Delta x = 1m \quad (8)$$

Setter $\Delta x = 1m$ og $v_{max} = 28m/s$ i CFL-betingelsen

$$\frac{\Delta t}{1m} \cdot 28m/s \leq 1 \quad (9)$$

$$\Rightarrow \Delta t \leq \frac{1m}{28m/s} \quad (10)$$

$$\Rightarrow \Delta t \leq \frac{1}{28}s \quad (11)$$

Legger inn margin etter som CFL-betingelsen er en maksgrense for stabil diskretisering.

$$\Delta t = 0.9 \cdot \frac{1}{28} s \quad (12)$$

Python-kode for å animere $u(x, t)$ over tid ved bruk av Lax-Friedrichs metode:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider

L = 1000 # banelengde
v_max = 28 # maksfart i m/s
Nx = 1000 # antall punkt langs den diskretiserte x-aksen
T_max = 600 # makstid
dx = 2 * L / Nx # delta x mellom hvert punkt
dt = 0.9 * (dx / v_max) # delta t mellom hvert punkt
Nt = int(T_max / dt)
x = np.linspace(-L, L, Nx, dtype=np.float64)

u_max = 0.1 # maksimal tetthet i biler/m

# gyldige p-verdier
valid_p = [1, 2, 5]

# hastighetsfunksjon
def v(u, p):
    return v_max * np.power(1 - (u / u_max), p)

# fluksfunksjon
def J(u, p):
    return u * v(u, p)

# beregn U(x, t) for alle p-verdier
U_all = {}
for p in valid_p:
    U_all[p] = np.zeros((Nt, Nx), dtype=np.float64)

for p in valid_p:
    u = np.zeros(Nx, dtype=np.float64)
    u[x < 0] = u_max # initialverdi

    U = np.zeros((Nt, Nx), dtype=np.float64)
    U[0, :] = u.copy()

    for n in range(Nt - 1):
        J_u = J(u, p)
        for i in range(1, Nx - 1):
            U[n + 1, i] = 0.5 * (u[i - 1] + u[i + 1]) - (dt / (2 * dx)) * (
                J_u[i + 1] - J_u[i - 1])
```

```

    U[n + 1, -1] = 0
    u[:] = U[n + 1, :]

    U_all[p] = U

# plotting og animering
fig, ax = plt.subplots(figsize=(8, 4))
plt.subplots_adjust(bottom=0.2)

colors = {1: 'b', 2: 'g', 5: 'r'}

lines = {}
for p in valid_p:
    lines[p] = ax.plot(x, U_all[p][0], colors[p], label=f'p = {p}')[0]

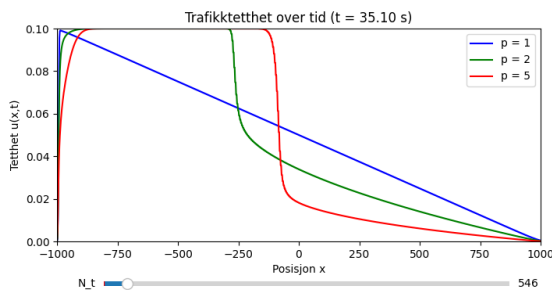
ax.set_xlim(-L, L)
ax.set_ylim(0, u_max)
ax.set_xlabel('Posisjon x')
ax.set_ylabel('Tetthet u(x,t)')
title = ax.set_title(f'Trafikktetthet over tid (t = 0.00 s)')
ax.legend()

ax_slider = plt.axes([0.2, 0.05, 0.65, 0.03])
slider = Slider(ax_slider, 'N_t', 0, Nt - 1, valinit=0, valstep=1)

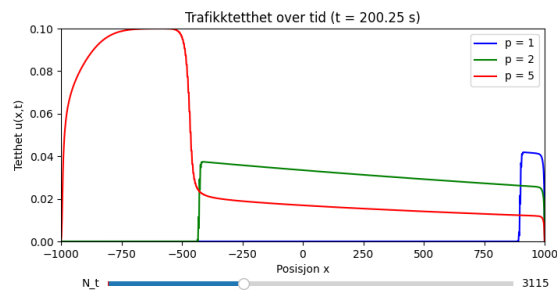
def update(val):
    frame = int(slider.val)
    for p in valid_p:
        lines[p].set_ydata(U_all[p][frame])
    title.set_text(f'Trafikktetthet over tid (t = {frame * dt:.2f} s)')
    fig.canvas.draw_idle()

slider.on_changed(update)
plt.show()

```



Figur 5: Tetthetsgraf for $t = 35.10$ s



Figur 6: Tetthetsgraf for $t = 200.25$ s

Animasjonen viser at ved start står alle biler i kø bak $x = 0$ med maksimal tetthet u_{max} og ingen biler der $x \geq 0$, noe som danner en sjokkbølge. Når tiden begynner å gå startes trafikkstrømmen og bilene nærmest $x = 0$ akselererer først, etterfulgt av bilene bak de. Her begynner forskjellene mellom de ulike p -verdiene.

- $p = 1$: Trafikken startet jevnere, sjokkbølgen spredde seg raskere og trafikken har kjørt gjennom banen relativt raskt.
- $p = 2$: Trafikken startet tregere, sjokkbølgen spredde seg saktere og trafikken har kjørt gjennom banen tregere.
- $p = 5$: Trafikken var meget treg, sjokkbølgen spredde seg sakte og trafikken kjørte gjennom banen lenge etter tilfellene der $p = 1$ og $p = 2$.

Høyere verdier for p fører til mer ujevn fordeling av tetthet u , etter som en høyere p -verdi øker hvor mye hastigheten v reduseres basert på tetthet u . Det vil si at en høyere p -verdi fører til tregere spredning av sjokkbølgen.

2.2.4 d) Bevegelsen til en enkelt bil

Vi ser på bevegelsen til en enkelt bil som starter ved $P(0) = 1000$ og beveger seg fremover etter trafikklyset har blitt grønt. Bevegelsen til bilen avhenger trafikk tettheten $u(x, t)$. Vi er gitt at:

- $P(0) = -1000$ og $V(0) = 0$
- $V(t) = v(u(P(t), t))$
- $P(t + \Delta t) = P(t) + \Delta t \cdot V(t)$

Og vi vet i tillegg at:

$$v(u) = v_{max} \left(1 - \left(\frac{u}{u_{max}} \right)^p \right)$$

Regner da ut $P(t)$ og $V(t)$ numerisk for de gitte p -verdiene og plotter de.

Python kode for numerisk utregning og plotting av grafene til $P(t)$ og $V(t)$ ved de gitte p -verdiene

```
import numpy as np
import matplotlib.pyplot as plt

L = 1000 # positiv banelengde
```

```

v_max = 28 # maksfart i m/s
Nx = 2000 # antall punkt langs den diskretiserte x-aksen
T_max = 600 # makstid
dx = 2 * L / Nx # delta x mellom hvert punkt
dt = 0.9 * (dx / v_max) # delta t mellom hvert punkt
Nt = int(T_max / dt) # antall tidspunkt
x = np.linspace(-L, L, Nx, dtype=np.float64)
u_max = 0.1 # maksimal tetthet i biler/m

valid_p = [1, 2, 5]

# hastighetsfunksjon
def v(u, p):
    return v_max * np.power(1 - (u / u_max), p)

# fluksfunksjon
def J(u, p):
    return u * v(u, p)

# regner ut U, P og V med en gitt p-verdi
def compute_traffic(p):
    u = np.zeros(Nx, dtype=np.float64)
    u[x < 0] = u_max # initialverdi

    # initialiser U
    U = np.zeros((Nt, Nx), dtype=np.float64)
    U[0, :] = u.copy()

    # gjennomfør lax-friedrichs metode
    for n in range(Nt - 1):
        J_u = J(u, p)
        for i in range(1, Nx - 1):
            U[n + 1, i] = 0.5 * (u[i - 1] + u[i + 1]) - (dt / (2 * dx)) * (
                J_u[i + 1] - J_u[i - 1])
        U[n + 1, -1] = 0 # randbetingelse
        u = U[n + 1, :].copy()

    # bilbevegelsen
    dt_bil = 0.1 # tidssteg for bilen
    Nt_bil = int(T_max / dt_bil)
    t_bil = np.linspace(0, T_max, Nt_bil)
    P = np.zeros(Nt_bil)
    V = np.zeros(Nt_bil)
    P[0] = -1000 # startposisjon

    def find_nearest_index(value, array):
        return np.argmin(np.abs(array - value))

    # regn ut P(t) og V(t)
    for n in range(Nt_bil - 1):
        t = n * dt_bil # finner nåværende tidspunkt for bilen

```

```

    nt_U = min(int(t / dt),
                Nt - 1) # finner nærmest tilsvarende tidspunkt i U(x,t)
    u_p = U[nt_U, find_nearest_index(P[n], x)] # trafikk tetthet ved xP[n]
    V[n] = v(u_p, p)
    P[n + 1] = P[n] + dt_bil * V[n] # eulers metode

    # kutt grafen når x = 1000
    krysningspunkt = np.where(P >= 999)[0][0]
    t_bil = t_bil[:krysningspunkt + 1]
    P = P[:krysningspunkt + 1]
    V = V[:krysningspunkt + 1]

    return t_bil, P, V

results = {}
for p in valid_p:
    t_bil, P, V = compute_traffic(p)
    results[p] = (t_bil, P, V)

# plotting
fig, axs = plt.subplots(2, 1)
colors = {1: 'b', 2: 'g', 5: 'r'}

for p in valid_p:
    t_bil, P, V = results[p]
    label_p = f'p = {p}'

    # plott P(t)
    axs[0].plot(t_bil, P, color=colors[p], label=label_p)

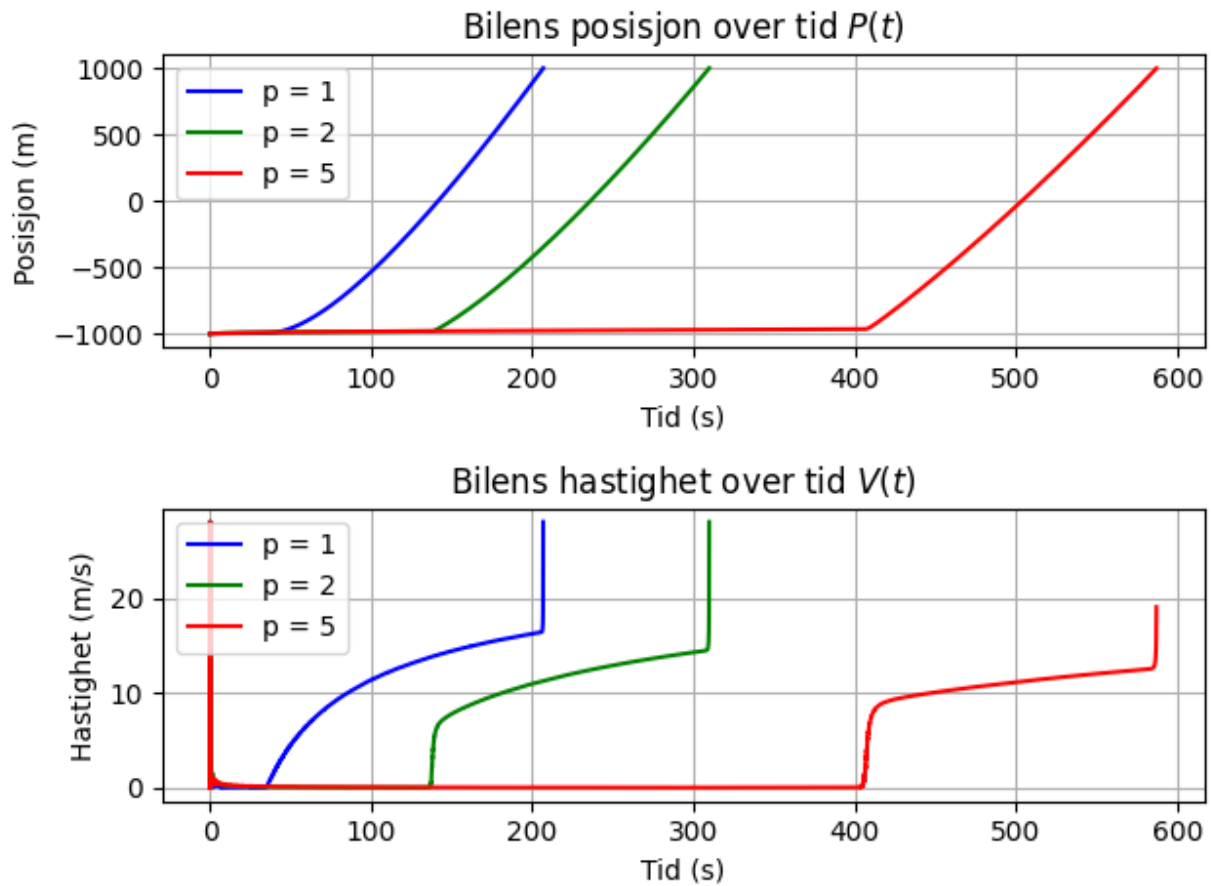
    # Plott V(t)
    axs[1].plot(t_bil, V, color=colors[p], label=label_p)

axs[0].set_xlabel('Tid (s)')
axs[0].set_ylabel('Posisjon (m)')
axs[0].set_title('Bilens posisjon over tid $P(t)$')
axs[0].legend()
axs[0].grid()

axs[1].set_xlabel('Tid (s)')
axs[1].set_ylabel('Hastighet (m/s)')
axs[1].set_title('Bilens hastighet over tid $V(t)$')
axs[1].legend()
axs[1].grid()

plt.tight_layout()
plt.show()

```

Figur 7: Oppgave 2d: Posisjon og hastighetsgrafer for den bakerste bilen

Hoppene i hastighet mot slutten av grafene skyldes at det nærmeste diskrete punktet på x-aksen som grafene klippes etter ligger litt over $x = 1000$ der randbetingelsen $u(1000, t) = 0$ gjelder.

2.2.5 e) Hastigheten til en kø

Python kode for utregning og plotting av $K(t)$

```
import numpy as np
import matplotlib.pyplot as plt

L = 1000 # positiv banelengde
v_max = 28 # maksfart i m/s
Nx = 2000 # antall punkt langs den diskretiserte x-aksen
T_max = 600 # makstid
dx = 2 * L / Nx # delta x mellom hvert punkt
dt = 0.9 * (dx / v_max) # delta t mellom hvert punkt
Nt = int(T_max / dt) # antall tidspunkt
x = np.linspace(-L, L, Nx, dtype=np.float64)
t = np.linspace(0, T_max, Nt)
u_max = 0.1 # maksimal tetthet i biler/m

valid_p = [1, 2, 5]
```

```

# hastighetsfunksjon
def v(u, p):
    return v_max * np.power(1 - (u / u_max), p)

# fluksfunksjon
def J(u, p):
    return u * v(u, p)

def compute_traffic(p):
    u = np.zeros(Nx, dtype=np.float64)
    u[x < 0] = u_max # initialverdi

    # initialiser U
    U = np.zeros((Nt, Nx), dtype=np.float64)
    U[0, :] = u.copy()

    # gjennomfør lax-friedrichs metode
    for n in range(Nt - 1):
        J_u = J(u, p)
        for i in range(1, Nx - 1):
            U[n + 1, i] = 0.5 * (u[i - 1] + u[i + 1]) - (dt / (2 * dx)) * (J_u[i + 1] - J_u[i - 1])
        U[n + 1, -1] = 0 # randbetingelse
        u = U[n + 1, :].copy()

    # K(t)
    K = np.zeros(Nt)
    for n in range(Nt - 1):
        first_moving_index = np.where(U[n, :] < u_max)[0][0]
        K[n] = x[first_moving_index]
    return K

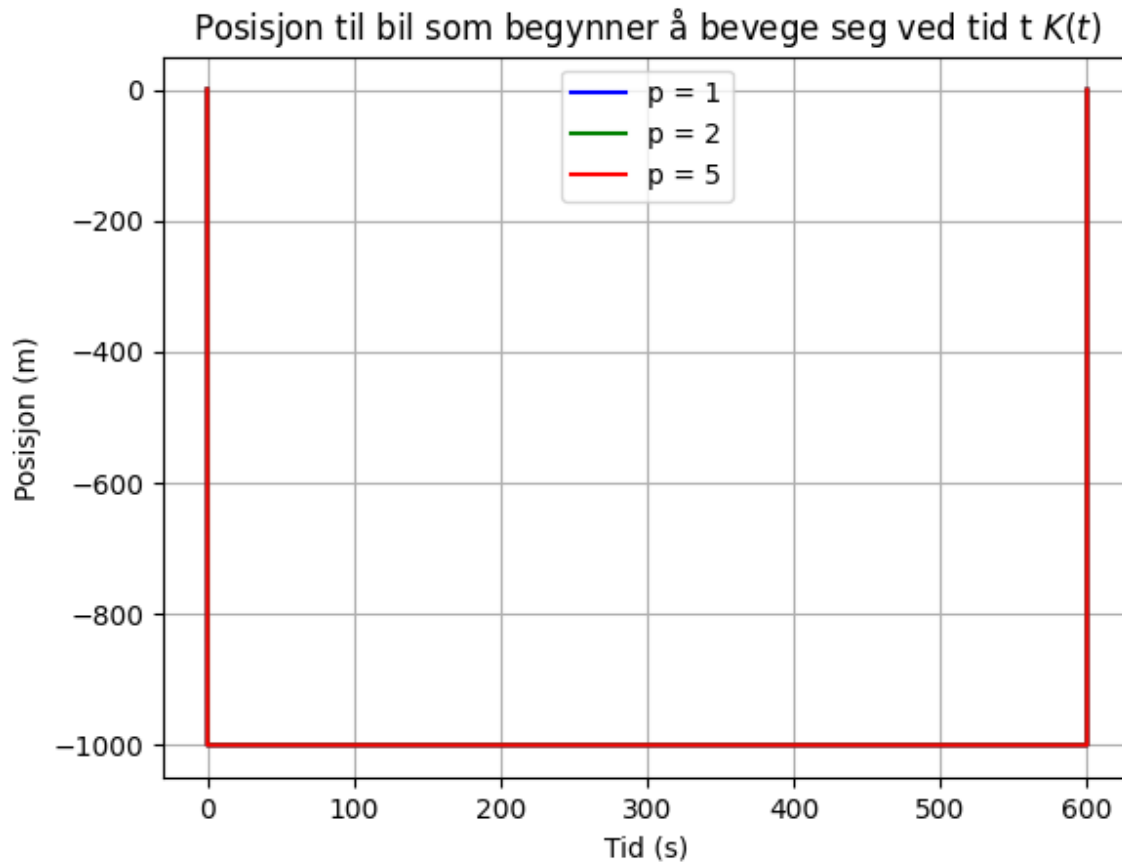
results = {}
for p in valid_p:
    K = compute_traffic(p)
    results[p] = K

colors = {1: 'b', 2: 'g', 5: 'r'}

# Plotting
plt.figure()
for p in valid_p:
    plt.plot(t, results[p], color=colors[p], label=f'p = {p}')

plt.xlabel('Tid (s)')
plt.ylabel('Posisjon (m)')
plt.title('Posisjon til bil som begynner å bevege seg ved tid t $K(t)$')
plt.legend()
plt.grid()
plt.show()

```



Figur 8: Oppgave 2e: Posisjon til bilen som begynner å bevege seg ved tid t

Ettersom hastigheten $V(t)$ er invers proporsjonal med tetthet $u(x, t)$, slik at når $u = u_{max}$ vil hastigheten til en bil være 0. For å finne posisjonen til den første bilen som begynner å bevege seg ved tiden t , altså $K(t)$, bør vi finne den laveste x -verdien der $u(x, t) < u_{max}$. Grunnen til at koden ovenfor ikke fungerer er på grunn av at tettheten synker ikke kun foran sjokkbølgen, men synker også litt nære $x = -1000$. Ved $t = 0$ har ikke bølgen beveget seg noe slik at den laveste x -verdien der $u(x, t) < u_{max}$ er $x = 0$, deretter vil $x = -1000$.

2.2.6 f) Valg av parameter p

- $p = 1$: For $p = 1$ forventer man at farten reduseres gradvis mot høyere tetthet og en lineær hurtighetsnedgang, fordi $v(u)$ er lineær når $p = 1$. Dette fører til raskere trafikkorkoppløsning, som man kan se i figur 4, der grafene for $p = 1$ er ferdig før de andre p -verdiene.
- $p = 2$: For $p = 2$ forventer man at hurtighetsnedgang er kvadratisk. $p = 2$ fører til skarpere hurtighetsendringer.
- $p = 5$: Ved høyere p -verdi kan man se at hurtighetsendringer blir bråere, og det tar lengre tid å løse opp trafikkorken.

3 Del 2: Varmeledning

3.1 Oppgave 3: Poissonligning, 1D

Oppgave A løser jeg analytisk og for hånd for å skape en variert løsning på denne oppgaven. Oppgave B løser jeg fullstendig i Python ved hjelp av koden gitt i "Uke 7". Dette er fordi det vil være vesentlig lettere å plote både punktene inn i Python enn for hånd. Løsningen er som vist nedenfor særdeles like hverandre som tyder på rett svar.

Ligningen:

$$u_{xx}(x) = f(x) \quad \text{der } -1 \leq x \leq 1, \quad \text{hvor } f(x) = \cos(\pi x)$$

Vi har også blitt gitt rand-betingelsene som er:

$$u(-1) = 0, \quad u(1) = 2.$$

$$\frac{d^2 u}{dx^2} = \cos(\pi x)$$

$$\Rightarrow \frac{du}{dx} = \int \cos(\pi x) dx$$

$$\Rightarrow \frac{du}{dx} = \frac{\sin(\pi x)}{\pi} + A$$

Nå integrerer vi igjen som følger:

$$u(x) = \int \left(\frac{\sin(\pi x)}{\pi} + A \right) dx$$

$$\Rightarrow u(x) = \frac{-\cos(\pi x)}{\pi^2} + Ax + B$$

Dette fører oss nærmere en generell løsning. Deretter gjør jeg som henvist til på referert plansje og finner A og B fra rand-betingelsne:

$$u(-1) = 0$$

$$\Rightarrow \frac{-\cos(-\pi)}{\pi^2} + (-A) + B = 0$$

$$\Rightarrow \frac{1}{\pi^2} - A + B = 0$$

$$u(1) = 2$$

$$\Rightarrow \frac{-\cos(\pi)}{\pi^2} + A + B = 2$$

$$\Rightarrow \frac{1}{\pi^2} + A + B = 2$$

Disse rand-betingelsene gir oss et lignings-system. Setter vi så inn ligning 1 i ligning 2 får vi som følger:

$$\left(\frac{1}{\pi^2} + A + B\right) - \left(\frac{1}{\pi^2} - A + B\right) = 2$$

Som da gir oss:

$$\Rightarrow 2A = 2$$

$$\Rightarrow A = 1$$

Setter vi så $A = 1$ inn i ligning nummer 2 får vi:

$$\Rightarrow \frac{1}{\pi^2} - 1 + B = 0$$

$$\Rightarrow B = 1 - \frac{1}{\pi^2}$$

Ved hjelp av rand-betingelses-settet kan vi derav løse original-ligningen analytisk og ender opp med følgende svar:

$$u(x) = \frac{-\cos(\pi x)}{\pi^2} + x + 1 - \frac{1}{\pi^2}$$

3.1.1 Numerisk løsning av oppgave 3

```
import numpy as np
import matplotlib.pyplot as plt

antall = 1000
x_verdier = np.linspace(-1, 1, antall)
h = x_verdier[1] - x_verdier[0]
A = np.zeros((antall, antall))
b = np.cos(np.pi * x_verdier)

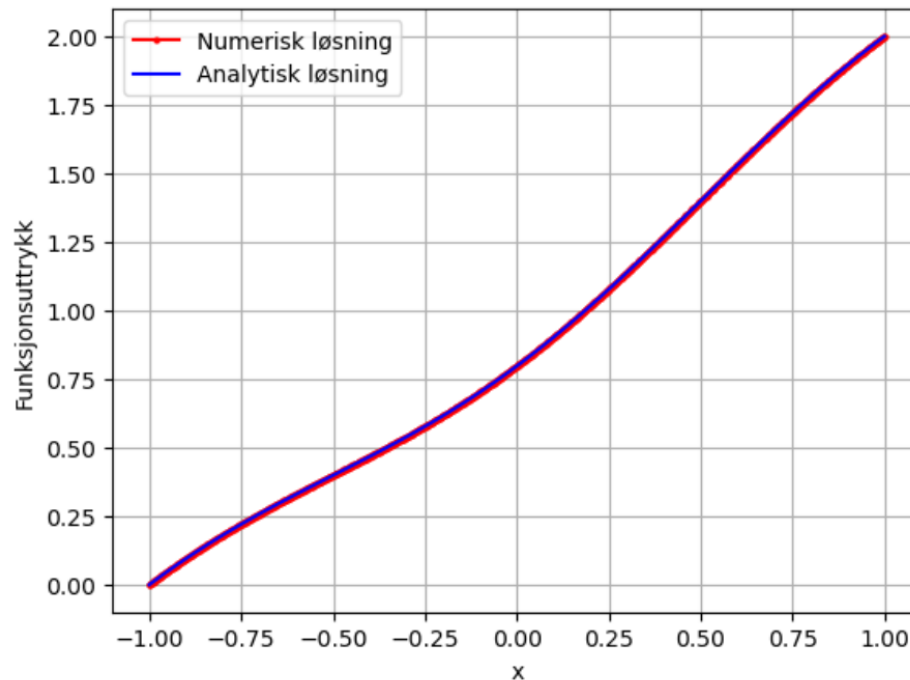
for i in range(1, antall - 1):
    A[i, i - 1] = 1 / h**2
    A[i, i] = -2 / h**2
    A[i, i + 1] = 1 / h**2

A[0, 0] = 1
b[0] = 0
A[-1, -1] = 1
b[-1] = 2

numerisk_losning = np.linalg.solve(A, b)
analytisk_losning = -1/np.pi**2 * np.cos(np.pi * x_verdier) + x_verdier + 1 - 1/np.pi**2

plt.plot(x_verdier, numerisk_losning, 'ro-', markersize=2, label="Numerisk løsning")
plt.plot(x_verdier, analytisk_losning, 'b-', label="Analytisk løsning")
plt.xlabel("x")
plt.ylabel("Funksjonsuttrykk")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



Figur 9: Oppgave 3: Sammenligning av A. og N. løsning

MathWorks (2025) James McKernan (2012)

3.2 Oppgave 4: Varmeligning, 1D

```
import numpy as np
import matplotlib.pyplot as plt

minimums_x, maximums_x = -1, 1
storste_t_verdi = 1.0
antall_x = 100
part_deriv_x = (maximums_x - minimums_x) / (antall_x - 1)
part_deriv_t = (0.4 * part_deriv_x**2)
t = int(np.ceil(storste_t_verdi / part_deriv_t))
x = np.linspace(minimums_x, maximums_x, antall_x)
liste_t_verdier = [0, 0.05, 0.075, 1] # Ulike t - verdier for å vise temperaturfordeling over tid

#Varmeligningen
def f(x):
    return np.cos(np.pi * x)

#Startsbetingelsen
def start(x):
    return 1 + x + 5 * np.sin(np.pi * x)

#Rand-betingelsene
u = np.zeros((t + 1, antall_x))
```

```

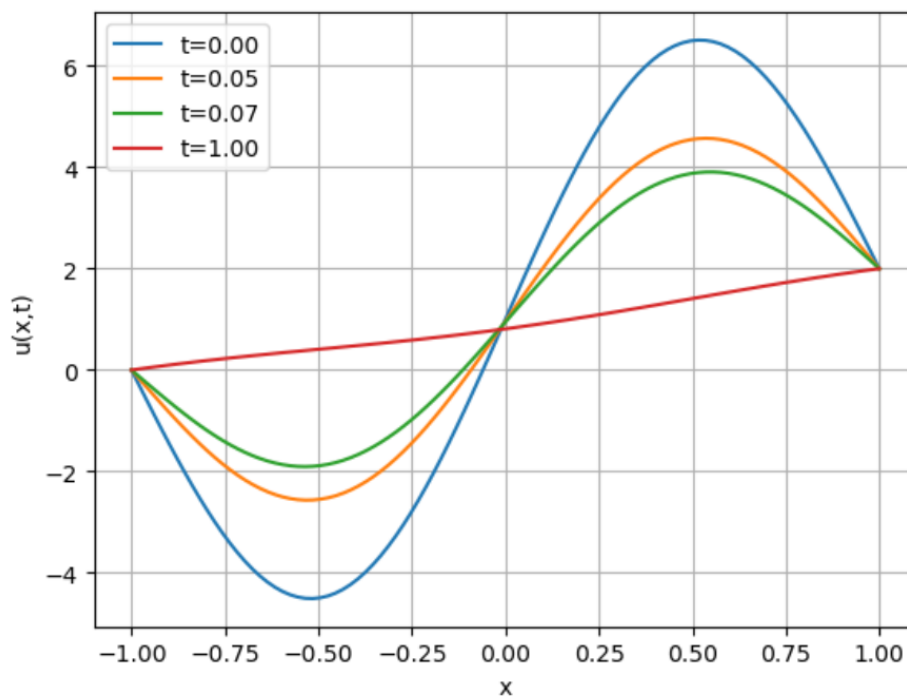
u[0, :] = start(x)
u[:, 0] = 0
u[:, -1] = 2

for n in range(t):
    for i in range(1, antall_x - 1):
        u[n+1, i] = u[n, i] + part_deriv_t * ((u[n, i+1] - 2*u[n, i] + u[n, i-1]) / part_deriv_x**2 - f(x[i]))
    u[n+1, 0] = 0
    u[n+1, -1] = 2

#Lager en for loop for å kunne endre og teste ulike t verdier /
for t_verdier in liste_t_verdier:
    t_tull = min(t, max(0, int(t_verdier / part_deriv_t)))
    plt.plot(x, u[t_tull, :], label=f't={t_verdier:.2f}')

plt.xlabel("x")
plt.ylabel("u(x,t)")
plt.legend()
plt.grid(True)
plt.show()

```



Figur 10: Oppgave 4: Varmeligningen med ulike T-verdier for å vise varmfordeling over tid

G. Nervadof (2021)

Figuren ovenfor viser et visualisert plot av Python-koden. Plottet inneholder 4 grafer med 4 forskjellige T(tidspunkt)-Verdier. Vi tenkte oss fram til at både oppgaven og ligningen vil bli enklere å forstå dersom vi la inn en liste med ulike tidsverdier i koden for å vise hvordan en varmeligning utvikler seg i forhold til både X og T verdier.

Grunnlaget i denne distinksjonen er at original ligningen med opprinnelig startsbetingelse ikke tar hensyn til T-verdiene. Startsbetingelsen gir oss i bunn og grunn grafen til varmeligningen før noe skjer; altså ved tid:

$T=0$. Ved å legge til flere tidspunkter er det til hjelp med å forme forståelsen av at temperatur faktisk ikke bare forandrer seg over tid men også fordeling av varmen på et element.

I vårt tilfelle har vi prøvd å konkretisere ligningen til et mer virkelighets-nært eksempel. Vi har visualisert det slik i oppgave at denne ligningen viser hvordan temperaturen er fordelt ovenfor en aksling over tid. Uansett om en side blir eller har blitt varmet opp vil motsatt side av akslingen være kald ved $T = 0$.

Over tid vil gjennomsnittstemperaturen i selve objektet stige, og derav vil temperaturen også fordele seg mer. Med andre ord vil total temperatur forbli det samme (eventuelt varmetap), men den vil være fordelt annerledes over samme areal/objekt. Viser vi til plottet ser vi det ved at i $T = 0$ er det store ytterpunkter i temperaturdelta, men ser vi videre på $T = 1$ ser ligningen betrakelig mer lineær ut og varmen har derfor blitt fordelt mer utover samme areal. Vi nærmer oss altså temperaturisk likevekt.

3.3 Oppgave 5: Poissonligning, 2D

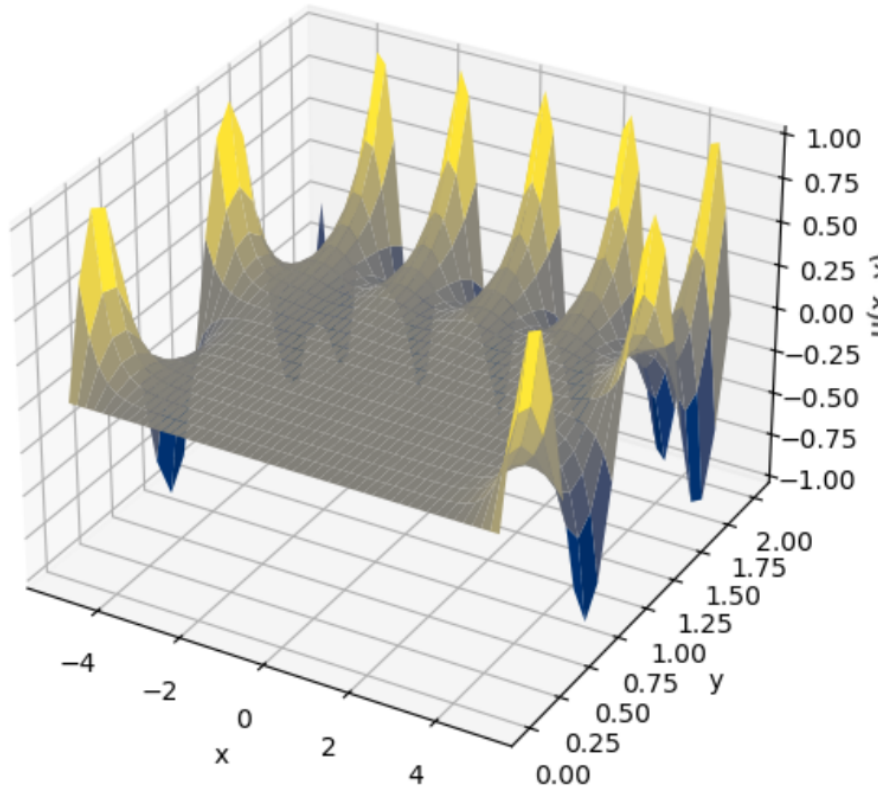
```
import numpy as np
import matplotlib.pyplot as plt

antall_x, antall_y = 50, 20
x = np.linspace(-5, 5, antall_x)
y = np.linspace(0, 2, antall_y)
X, Y = np.meshgrid(x, y)
u = np.zeros((antall_x, antall_y))

#Rand-betingelsene
u[0, :] = np.sin(2 * np.pi * y)
u[-1, :] = np.sin(2 * np.pi * y)
u[:, 0] = 0
u[:, -1] = np.sin(np.pi * x)

for i in range(5000):
    u[1:-1, 1:-1] = 0.25 * (u[:-2, 1:-1] + u[2:, 1:-1] + u[1:-1, :-2] + u[1:-1, 2:])

fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X, Y, u.T, cmap="cividis", edgecolor="none")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("u(x, y)")
plt.show()
```

Figur 11: Oppgave 5: Poissonligning, 2D, visualisert

3.4 Oppgave 6: Varmeligning

3.4.1 a)

Vi velger en gjenstand laget av stål, som har en termisk diffusivitet på (Engineersedge, 2025)

$$\alpha = 1.172 \times 10^{-5} \text{ m}^2/\text{s}$$

Gjenstanden har et rektangulært tverrsnitt med dimensjoner:

$$x = 0.4 \text{ m}, \quad y = 0.28 \text{ m}.$$

3.4.2 b)

Den partielle differensialligningen for varmeledning er gitt som:

$$u_t = \alpha \cdot (u_{xx} + u_{yy})$$

hvor $\alpha = 1.172 \times 10^{-5} \text{ m}^2/\text{s}$

Randbetingelser: Temperaturen på kantene av legemet holdes konstant på 200°C :

$$u(0, y, t) = u(0.4, y, t) = u(x, 0, t) = u(x, 0.28, t) = 200$$

Initialbetingelse: Ved $t = 0$ er temperaturen i hele legemet 20°C (vi antar at 20°C er romtemperatur):

$$u(x, y, 0) = 20$$

Numerisk løsning:

Vi bruker en endelig differansemetode for å løse differensialligningen numerisk. Rommet diskretiseres med $nx = 40$ punkter i x -retning og $ny = 28$ punkter i y -retning. Tidssteglengden er gitt av:

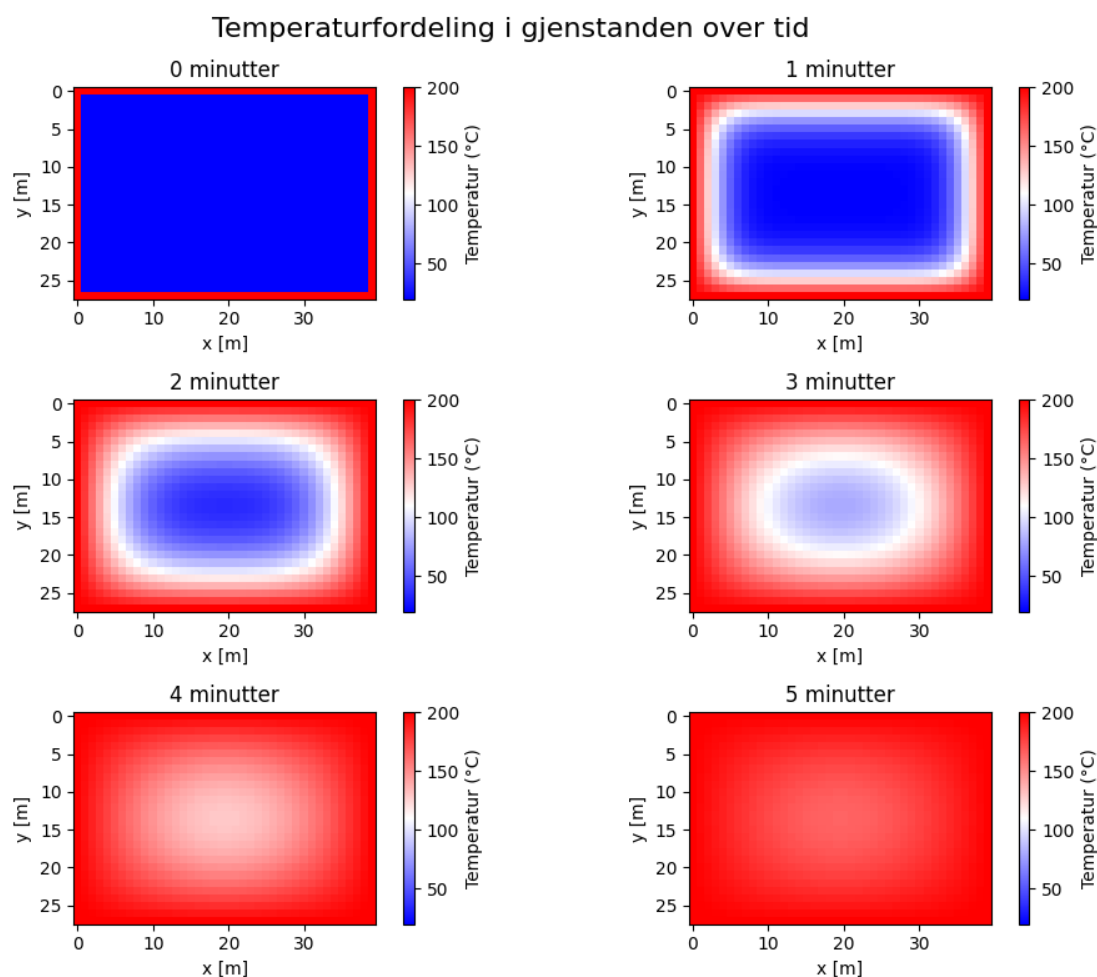
$$dt = 0.5 \cdot \frac{dx^2 + dy^2}{\alpha * (dx^2 + dy^2)}$$

hvor $dx = 0.01$ m og $dy = 0.01$ m.

Temperaturen oppdateres ved hvert tidssteg ved hjelp av (Scipython, 2025):

$$u_{i,j}^{n+1} = u_{i,j}^n + \alpha \cdot dt \cdot \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{dx^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{dy^2} \right)$$

Figur 12 nede viser hvordan temperaturen sprer seg fra kantene mot midten av gjenstanden over tid.



Figur 12: Oppgave 6b: Temperaturfordeling i en gjenstand over tid.

```
import numpy as np
import matplotlib.pyplot as plt
# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
```

```

nx, ny = 40, 28 # Antall punkter i x- og y-retning

dx, dy = Lx / (nx), Ly / (ny) # Romlig steglengde
nx, ny = int(Lx/dx), int(Ly/dy)
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20 # Starttemperatur 20 grader

# Funksjon for å løse varmelikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()

        u[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )

        # Randbetingelser
        u[:, 0] = 200 # Venstre kant
        u[:, -1] = 200 # Høyre kant
        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant

    return u

# Løsning og plotting ved ulike tidspunkt
times = [0, 1*60, 2*60, 3*60, 4*60, 5*60] # Tid i sekunder
titles = ["0 minutter", "1 minutter", "2 minutter", "3 minutter", "4 minutter", "5 minutter"]
fig, axes = plt.subplots(3, 2, figsize=(10, 8)) # 3x2 grid av plott
fig.suptitle("Temperaturfordeling i legemet over tid", fontsize=16)

# Løsning og plotting ved ulike tidspunkt
for i, t in enumerate(times):
    steps = int(t / dt)
    u = solve_heat_equation(u, alpha, dx, dy, dt, steps)

    # plotting
    ax = axes[i // 2, i % 2]
    im = ax.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
    ax.set_title(titles[i])
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')

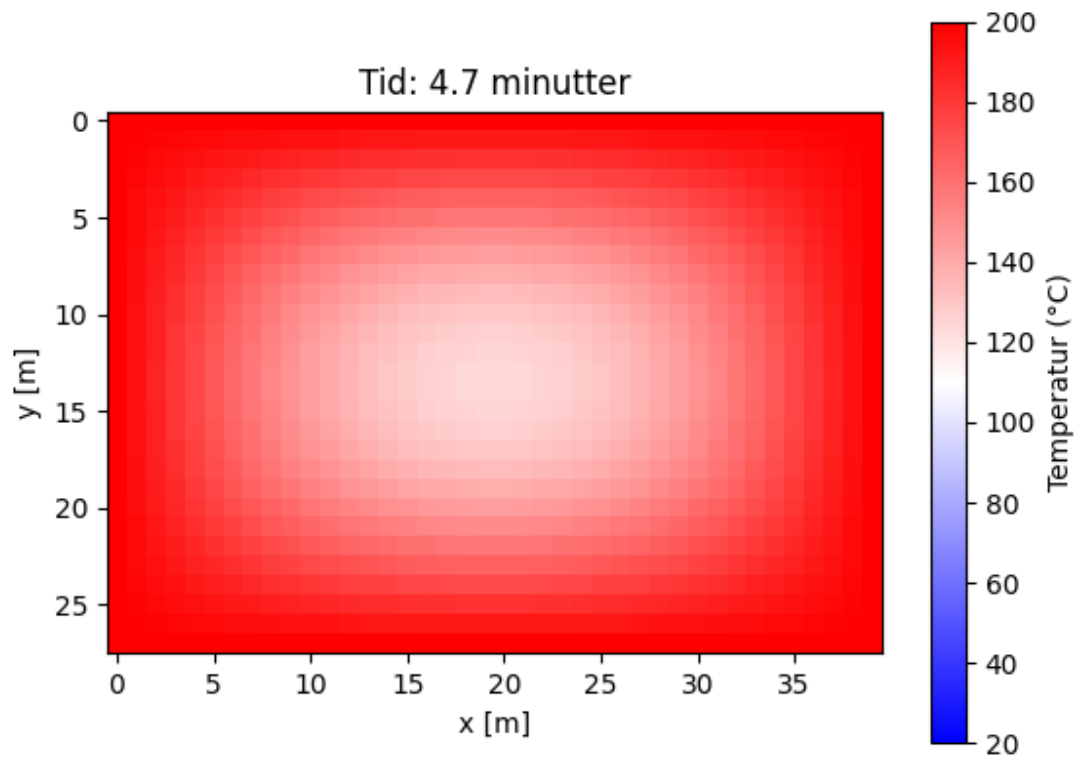
    fig.colorbar(im, ax=ax, label='Temperatur (°C)')

plt.tight_layout()
plt.show()

```

3.4.3 c)

Vi simulerer temperaturen i legemet over tid og finner at temperaturen i midten $(x, y) = (0.2, 0.14)$ når 60°C etter ca. (4.7 minutter). Figur 13 viser temperaturfordelingen ved dette tidspunktet.



Figur 13: Oppgave 6c: Temperaturfordeling ved $t \approx 4.7$ minutt

```
import numpy as np
import matplotlib.pyplot as plt

# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
nx, ny = 40, 28 # Antall punkter i x- og y-retning
dx, dy = Lx / (nx - 1), Ly / (ny - 1) # Romlig steglengde
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20

# Funksjon for å løse varmeledningslikningen
def solve_heat_equation2(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        u[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
    # Randbetingelser
    u[:, 0] = 200 # Venstre kant
    u[:, -1] = 200 # Høyre kant
```

```

        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant
    return u

# Funksjon for å finne når midten når 60 grader
def find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp):
    steps = 0
    mid_x, mid_y = nx // 2, ny // 2 # Midten av legemet
    while u[mid_x, mid_y] < target_temp:
        steps += 1
        u = solve_heat_equation(u, alpha, dx, dy, dt, 1) # Oppdater ett tidssteg
    return steps * dt

# Finn tiden
target_temp = 60
time_to_reach = find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp)
print(f"Det tar {time_to_reach/60:.1f} minutter å nå {target_temp}°C i midten.")

# Plot temperaturfordelingen ved dette tidspunktet
steps = int(time_to_reach / dt)
u = solve_heat_equation(u, alpha, dx, dy, dt, steps)
plt.figure()
plt.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
plt.colorbar(label='Temperatur (°C)')
plt.title(f'Tid: {time_to_reach/60:.1f} minutter')
plt.xlabel('x [m]')
plt.ylabel('y [m]')
plt.show()

```

3.4.4 d)

Vi lager en animasjon som viser hvordan temperaturen i legemet endrer seg over tid. Dette gjøres ved å oppdatere temperaturfordelingen ved hvert tidssteg og vise resultatet som en sekvens av bilder. Koden for animasjonen er gitt nedenfor.

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
%matplotlib notebook

# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
nx, ny = 40, 28 # Antall punkter i x- og y-retning
dx, dy = Lx / (nx - 1), Ly / (ny - 1) # Romlig steglengde
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20

# Funksjon for å løse varmeledningslikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        u[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
    return u

```

```

    # Randbetingelser
    u[:, 0] = 200 # Venstre kant
    u[:, -1] = 200 # Høyre kant
    u[0, :] = 200 # Nedre kant
    u[-1, :] = 200 # Øvre kant
    return u

# Opprett figuren
fig, ax = plt.subplots()
im = ax.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
plt.colorbar(im, label='Temperatur (°C)')
ax.set_xlabel('x [m]')
ax.set_ylabel('y [m]')
ax.set_title('Temperaturfordeling over tid')

# Funksjon for å oppdatere animasjonen
def update(frame):
    global u
    u = solve_heat_equation(u, alpha, dx, dy, dt, 10) # Oppdater 10 tidssteg per frame
    im.set_array(u.T)
    ax.set_title(f'Tid: {frame * dt * 10 / 60:.1f} minutter')
    return im,

# Lag animasjonen
ani = animation.FuncAnimation(fig, update, frames=100, interval=50, blit=True, repeat=False)
plt.show()

```

3.5 Oppgave 7: Luften skal med

For en mer realistisk måling, må vi regne med at temperaturen i ovnen vil gå ned når et kaldere legeme settes inn i ovnen. Vi inkluderer et lag av luft rundt legemet i modellen, men ignorerer konveksjon.

Vi setter opp beregningene fra oppgave 6 med utgangspunkt i at luftlaget ligger på temperaturen 200 grader når $t = 0$. Temperaturen til legemet vi setter inn ved $t = 0$ er 15 grader.

3.5.1 a)

Matematisk modell

Vi modellerer temperaturutviklingen i et legeme omgitt av et luftlag ved hjelp av varmelikningen i to dimensjoner:

$$u_t = \alpha \cdot (u_{xx} + u_{yy}), \quad (13)$$

hvor $u(x, y, t)$ er temperaturen som funksjon av tid og romlige koordinater, og $\alpha(x, y)$ er den termiske diffusiviteten, som varierer avhengig av materialet:

$$\alpha(x, y) = \begin{cases} \alpha_{\text{stål}}, & \text{for } (x, y) \text{ inne i legemet,} \\ \alpha_{\text{luft}}, & \text{for } (x, y) \text{ i luftlaget.} \end{cases} \quad (14)$$

Randbetingelser

Temperaturen holdes konstant på ytterkantene av luftlaget:

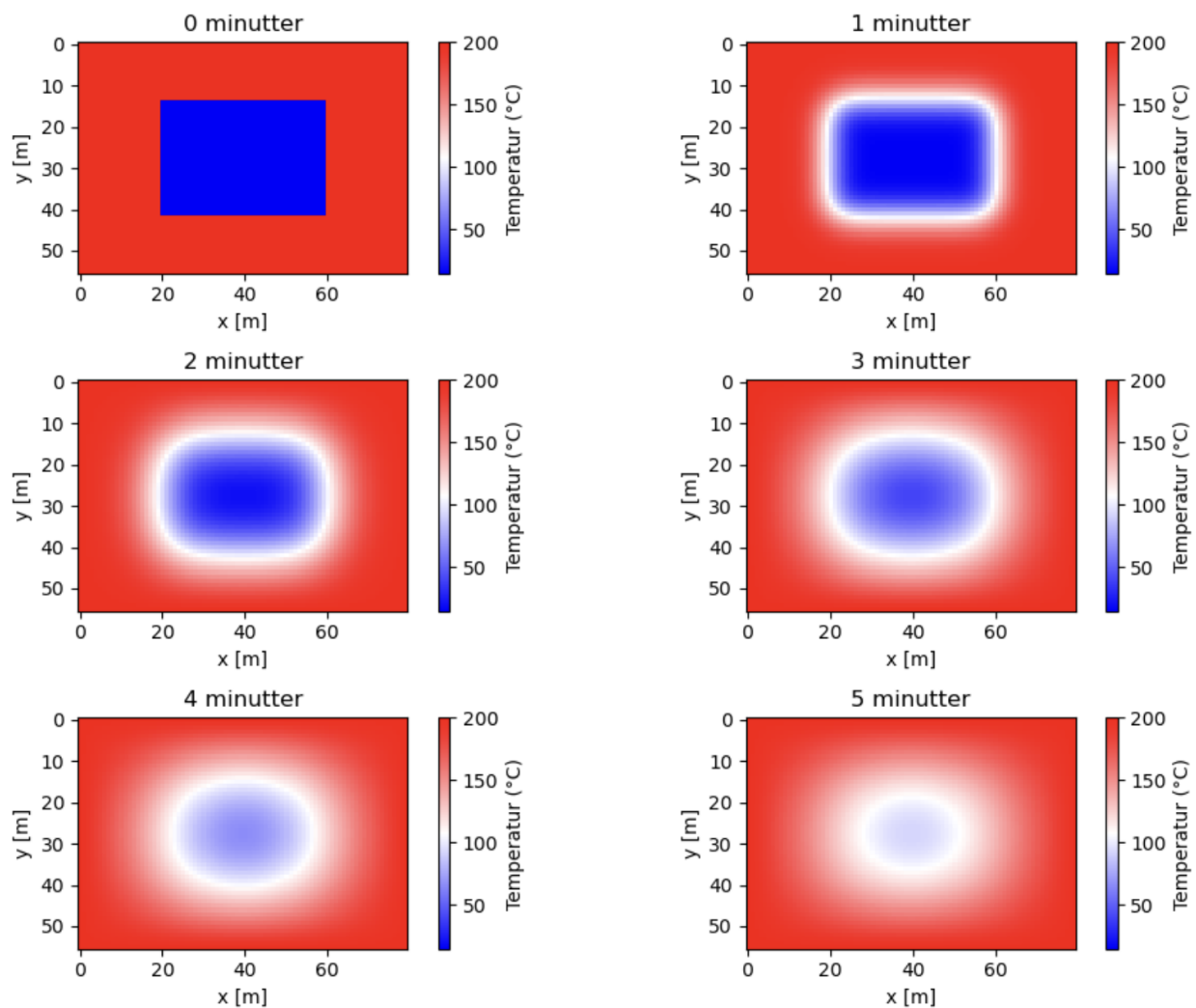
$$u(0, y, t) = u(L_x, y, t) = u(x, 0, t) = u(x, L_y, t) = 200^\circ\text{C}, \quad \forall t > 0. \quad (15)$$

Initialbetingelser

Ved $t = 0$ er temperaturen satt til:

$$u(x, y, 0) = \begin{cases} 15^\circ C, & \text{for } (x, y) \text{ inne i legemet,} \\ 200^\circ C, & \text{for } (x, y) \text{ i luftlaget.} \end{cases} \quad (16)$$

Temperaturfordeling i legemet og luftlaget



Figur 14: Oppgave 7: Varmeplot av temperaturfordelingen i legemet og luftlaget

```
import numpy as np
import matplotlib.pyplot as plt

# Parametere
alpha_steel = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
alpha_air = 2.0e-5 # Termisk diffusivitet for luft [m^2/s]

Lx, Ly = 0.8, 0.56 # Dimensjoner for hele ovnen (legeme + luft)
```

```

L_body_x, L_body_y = 0.4, 0.28 # Dimensjoner av selve legemet

dx, dy = 0.01, 0.01 # Romlig steglengde
nx, ny = int(Lx / dx), int(Ly / dy)

dt = dx**2 * dy**2 / (2 * alpha_air * (dx**2 + dy**2)) # Tidssteg

# Størrelse på legemet
body_x_start, body_x_end = int(0.2 / dx), int((0.2 + L_body_x) / dx)
body_y_start, body_y_end = int(0.14 / dy), int((0.14 + L_body_y) / dy)

# Temperaturmatrise
u = np.ones((nx, ny)) * 200
u[body_x_start:body_x_end, body_y_start:body_y_end] = 15 # Legemet starter på 15°C

alpha = np.ones((nx, ny)) * alpha_air
alpha[body_x_start:body_x_end, body_y_start:body_y_end] = alpha_steel

# Funksjon for å løse varmelikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        un[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha[1:-1, 1:-1] * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
        # Randbetingelser
        u[:, 0] = 200 # Venstre kant
        u[:, -1] = 200 # Høyre kant
        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant
    return u

# Løsning og plotting ved ulike tidspunkt

times = [0, 1*60, 2*60, 3*60, 4*60, 5*60] # Tid i sekunder
titles = ["0 minutter", "1 minutter", "2 minutter", "3 minutter", "4 minutter", "5 minutter"]
fig, axes = plt.subplots(3, 2, figsize=(10, 8)) # 3x2 grid av plott
fig.suptitle("Temperaturfordeling i legemet og luftlaget", fontsize=16)

for i, t in enumerate(times):
    steps = int(t / dt)
    u = solve_heat_equation(u, alpha, dx, dy, dt, steps)

    ax = axes[i // 2, i % 2]
    im = ax.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
    ax.set_title(titles[i])
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')
    fig.colorbar(im, ax=ax, label='Temperatur (°C)')

plt.tight_layout()
plt.show()

```

3.5.2 b)

Vi lager en animasjon som viser hvordan temperaturen til legemet endrer seg over tid. I starten ser vi observerer vi at temperaturendringene er raskest nær legemets ytterkanter, hvor det er i direkte kontakt

med det varme luftlaget. Dette skyldes at varmeledning skjer ved en temperaturgradient - varmen strømmer fra de varme områdene (luft på 200°C) mot de kaldere områdene (legemet på 60°C).

Oppvarmingen av legemet går imidlertid relativt sakte, noe som kan forklares ved at stål har lavere termisk diffusivitet enn luft. Termisk diffusivitet, α , bestemmer hvor raskt varme sprer seg gjennom et materiale og er gitt av:

$$\alpha = \frac{k}{\rho c_p}$$

hvor k er varmeledningsevnen, ρ er tettheten, og c_p er spesifikk varmekapasitet (Engineersedge, 2025). Luft har en mye høyere diffusivitet enn stål, noe som betyr at temperaturforandringer skjer raskere i luft enn i legemet. Dette resulterer i en tregere oppvarming av stålet sammenlignet med omgivelsene. I løpet av de første minuttene ser vi en tydelig temperaturgradient fra kantene og innover mot midten. Etter hvert som tiden går, begynner temperaturfordelingen i legemet å jevnnes ut, ettersom varmen forplanter seg videre inn mot sentrum. Rundt 10-minuttersmerket ser vi at kontrasten i temperaturfordelingen blir mindre tydelig – dette betyr at temperaturen nærmer seg en jevn fordeling, og at legemet nærmer seg en termisk likevekt med luftlaget. Mot slutten av simuleringen avtar temperaturendringene gradvis, ettersom forskjellen mellom temperaturen i legemet og luften minker.

```
# Oppretter figuren
fig, ax = plt.subplots()
im = ax.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
plt.colorbar(im, label='Temperatur (°C)')
ax.set_xlabel('x [m]')
ax.set_ylabel('y [m]')
ax.set_title('Temperaturfordeling over tid')

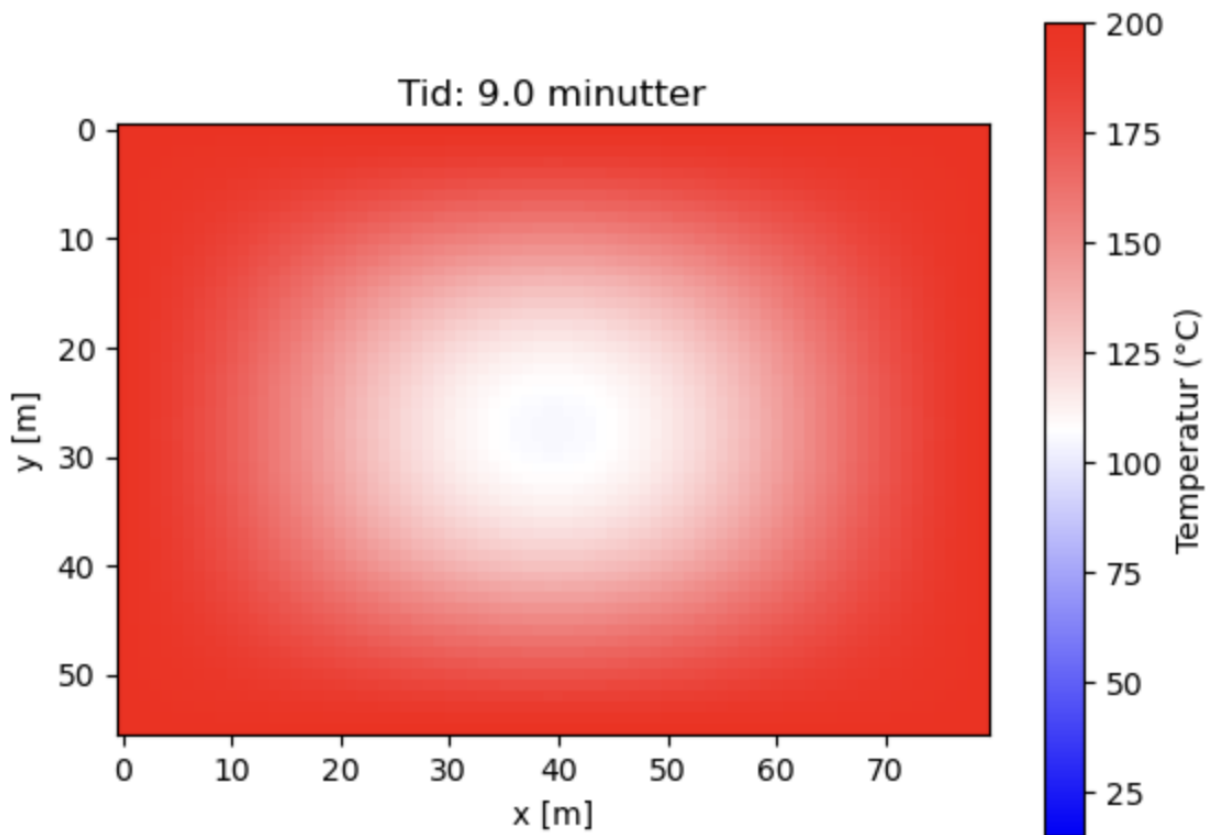
# Definerer animasjonen
def update(frame):
    global u
    u = solve_heat_equation(u, alpha, dx, dy, dt, 10) # Oppdater 10 tidssteg per frame
    im.set_array(u.T)
    ax.set_title(f'Tid: {frame * dt * 10 / 60:.1f} minutter')
    return im,

# Lager animasjonen
ani = animation.FuncAnimation(fig, update, frames=100, interval=50, blit=True, repeat=False)
plt.show()
```

3.5.3 c)

Som i oppgave 6, simulerer vi temperaturutviklingen i legemet over tid ved hjelp av den eksplisitte finitte differansemetoden (FTCS). Vi undersøker spesielt temperaturen i midten av legemet, definert som punktet $(x_{\text{midt}}, y_{\text{midt}})$, for å finne hvor lang tid det tar før denne når 60°C.

Ved å iterere over tidsstegene i simuleringen, registrerer vi temperaturen i dette punktet og finner at den når 60°C etter omtrent 9 min. Dette resultatet stemmer godt overens med forventningene, gitt at varmeledning i stål skjer relativt sakte sammenlignet med luft, og at midten av legemet er lengst unna de ytre kantene som varmes opp først. I begynnelsen av oppvarmingsprosessen er temperaturendringene raskest, ettersom temperaturforskjellen mellom legemet og luften er størst. Dette innebærer at temperaturøkningen i midten avtar etter hvert, ettersom temperaturen nærmer seg en stabil likevekt med omgivelsene. Figur 15 viser temperaturfordelingen i legemet ved det tidspunktet hvor midten når 60°C.



Figur 15: Oppgave 7: Varmeplot som viser tidspunktet hvor midten av legemet når 60°C

```
import numpy as np
import matplotlib.pyplot as plt

# Funksjon for å finne når midten når 60 grader
def find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp, body_x_start, body_x_end,
    ↪ body_y_start, body_y_end):
    steps = 0
    mid_x, mid_y = (body_x_start + body_x_end) // 2, (body_y_start + body_y_end) // 2
    while u[mid_x, mid_y] < target_temp:
        steps += 1
        u = solve_heat_equation2(u, alpha, dx, dy, dt, 1) # Oppdater ett tidssteg
    return steps * dt

# Finn tiden
target_temp = 60
time_to_reach = find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp, body_x_start,
    ↪ body_x_end, body_y_start, body_y_end)
print(f"Det tar {time_to_reach/60:.1f} minutter å nå {target_temp}°C i midten.")

# Plot temperaturfordelingen ved dette tidspunktet
steps = int(time_to_reach / dt)
u = solve_heat_equation2(u, alpha, dx, dy, dt, steps)
plt.figure()
plt.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
plt.colorbar(label='Temperatur (°C)')
plt.title(f'Tid: {time_to_reach/60:.1f} minutter')
```

```
plt.xlabel('x [m]')
plt.ylabel('y [m]')
plt.show()
```

3.6 Oppgave 8: Oppsummering av gruppearbeidet

Å jobbe med denne gruppeoppgaven har vært en positiv opplevelse både faglig og sosialt. Vi ble allerede på første møte enige om overordnede mål og hvor store ambisjoner vi hadde. Vi fordelte oppgavene så rettferdig som mulig mellom oss, og satte interne frister for når de forskjellige elementene skulle være fullført. Dette ga oss en tydelig plan å følge, og bidro til mindre stress i innspurten av prosjektet. For å sikre at alle var komfortable med arbeidsfordelingen diskuterte vi gruppemedlemmenes styrker og svakheter, og ble enige om å holde tett kommunikasjon og alltid være behjelpelig der det skulle trenge underveis i oppgavene, uansett om det var med matematikken, koding eller selve skrivingen i rapporten.

For å holde en strukturert plan ble vi enige om å holde ukentlige møter hver mandag, hvor vi diskuterte fremgang og eventuelle utfordringer som hadde dukket opp siden sist. Disse møtene hjalp på motivasjon underveis, og var en god plattform for å hjelpe hverandre med problemer vi hadde møtt på. Møtene hjalp oss holde fokus underveis i rapportskrivingen, og holdt oss oppdatert på hverandres status i arbeidsprosessen.

Vi ble tidlig i prosjektet enige om hvilke digitale verktøy vi skulle bruke, og endte opp med Microsoft Teams for kommunikasjon og Word for enklere dokumentskriving. Vi brukte også Jupyter Notebook gjennom NTNUs JupyterHub individuelt under oppgaveløsning og Overleaf for den endelige rapporten. Flere i gruppa var uerfarne med sistnevnte plattform, men brukte det som en måte å lære seg LaTeX på. Bruk av Overleaf ga oss god struktur og kontroll på formateringen i rapporten, tross en bratt læringskurve for de uten erfaring med verktøyet. Også her var gruppemedlemmer til stor hjelp, og vi lærte mye av hverandre.

Alt i alt har vi opplevd gruppearbeidet som lærerikt og motiverende, og føler vi har vært gjennom en prosess som både har forbedret kunnskap og samarbeidsevner. Vi har fått en verdifull erfaring i å arbeide strukturert i et team, og håper vi tar med oss dette videre inn i fremtidige prosjekt- og gruppeoppgaver både i studier og arbeidslivet.

Referanser

- Engineersedge. Thermal diffusivity table, 2025. URL https://www.engineersedge.com/heat_transfer/thermal_diffusivity_table_13953.htm. Last accessed 09 march 2025.
- G. Nervadof. Solving 2d heat equation numerically using python, 2021. URL <https://levelup.gitconnected.com/solving-2d-heat-equation-numerically-using-python-3334004aa01a>.
- James McKernan. Lecture 15 notes for 18.02 — MIT OpenCourseWare, 2012. URL https://math.mit.edu/~mckernan/Teaching/12-13/Autumn/18.02/1_15.pdf.
- MathWorks. Solve PDEs using PDEModel — MathWorks documentation, 2025. URL <https://se.mathworks.com/help/pde/ug/pde.pdemodel.solvepde.html>.
- Per Morten Merg. Vil ha bredere parkeringsplasser: Så mye har bilene «vokst» i norge. *Bilbransje24*, 2023.
- Scipython. The two-dimensional diffusion equation, 2025. URL <https://scipython.com/book/chapter-7-matplotlib/examples/the-two-dimensional-diffusion-equation/>. Last accessed 09 march 2025.
- Wikipedia. Courant–friedrichs–lewy condition, a. URL https://en.wikipedia.org/wiki/Courant%E2%80%93Friedrichs%E2%80%93Lewy_condition. Last accessed 10 march 2025.
- Wikipedia. Lax–friedrichs method, b. URL https://en.wikipedia.org/wiki/Lax%E2%80%93Friedrichs_method. Last accessed 10 march 2025.

4 Vedlegg A: Systematiserte kodeblokker til oppgavene

Vedlegg med kode tilhørende de ulike oppgavene i prosjektet. Systematisert for leselighet, uten sideskift.

4.1 Del 1

4.1.1 Oppgave 1b-iii

```
# Plott av den logaritmiske funksjonen
import numpy as np
import matplotlib.pyplot as plt

def v_log(u, v_max, u_max):
    """
    Logaritmisk hastighetsfunksjon.

    Args:
        u: Tetthet (array eller enkeltverdi).
        v_max: Maksimal hastighet.
        u_max: Maksimal tetthet.

    Returns:
        Hastighet (array eller enkeltverdi).
    """
    if isinstance(u, (int, float)):
        if u >= u_max:
            return 0
        else:
            return v_max * np.log((u_max + 1) / (u + 1)) / np.log(u_max + 1)
    else:
        v = np.zeros_like(u, dtype=float)
        mask = u < u_max
        v[mask] = v_max * np.log((u_max + 1) / (u[mask] + 1)) / np.log(u_max + 1)
        return v

# Definerer parametre
v_max = 60 # km/t
u_max = 100 # biler/km

# Lager et array med tetthetsverdier
u_values = np.linspace(0, u_max, 400)

# Beregner hastighetene
v_values = v_log(u_values, v_max, u_max)

# Tegner grafen
plt.figure(figsize=(8, 6))
plt.plot(u_values, v_values)
plt.xlabel("Tetthet (u) [biler/km]")
plt.ylabel("Hastighet (v) [km/t]")
plt.title("Logaritmisk hastighetsfunksjon")
plt.grid(True)
plt.xlim(0, u_max)
plt.ylim(0, v_max * 1.1)
plt.show()
```

4.1.2 Oppgave 1b-iii

```
# Plott av cosinus-funksjonen
import numpy as np
import matplotlib.pyplot as plt

def v_cos(u, v_max, u_max):
    """
    Cosinus-basert hastighetsfunksjon.

    Args:
        u: Tetthet (array eller enkeltverdi).
        v_max: Maksimal hastighet.
        u_max: Maksimal tetthet.

    Returns:
        Hastighet (array eller enkeltverdi).
    """
    return v_max * np.cos((np.pi / 2) * (u / u_max))

# Definerer parametre
v_max = 60 # km/t
u_max = 100 # biler/km

# Lager et array med tetthetsverdier
u_values = np.linspace(0, u_max, 400)

# Beregner hastighetene
v_values = v_cos(u_values, v_max, u_max)

# Tegner grafen
plt.figure(figsize=(8, 6))
plt.plot(u_values, v_values)
plt.xlabel("Tetthet (u) [biler/km]")
plt.ylabel("Hastighet (v) [km/t]")
plt.title("Cosinus-basert hastighetsfunksjon")
plt.grid(True)
plt.xlim(0, u_max)
plt.ylim(0, v_max * 1.1)
plt.show()
```

4.1.3 Oppgave 2c

```
# Python-kode for å animere  $u(x,t)$  over tid ved bruk av Lax-Friedrichs metode
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider

L = 1000 # banelengde
v_max = 28 # maksfart i m/s
Nx = 200 # antall punkt langs den diskretiserte x-aksen
Nt = 2000 # antall punkt langs den diskretiserte tidsaksen
dx = 2 * L / Nx # delta x mellom hvert punkt
dt = 0.9 * (dx / v_max) # delta t mellom hvert punkt
x = np.linspace(-L, L, Nx, dtype=np.float64)

u_max = 0.1 # maksimal tetthet i biler/m
u = np.zeros(Nx, dtype=np.float64)
u[x < 0] = u_max # initialverdi

# inputvalidering for p
p = 0
valid_p = {1, 2, 5}
while p not in valid_p:
    p = int(input("Skriv inn ønsket p-verdi: "))

print(f"p = {p}")

# hastighetsfunksjon
def v(u):
    return v_max * np.power(1 - (u / u_max), p)

# fluksfunksjon
def J(u):
    return u * v(u)

#initialiser U
U = np.zeros((Nt, Nx), dtype=np.float64)
U[0, :] = u.copy()

# gjennomfør lax-friedrichs metode
for n in range(Nt - 1):
    J_u = J(u)
    for i in range(1, Nx - 1):
        U[n + 1, i] = 0.5 * (u[i - 1] + u[i + 1]) - (dt / (2 * dx)) * (
            J_u[i + 1] - J_u[i - 1])

    U[n + 1, -1] = 0
    for i in range(Nx):
        u[i] = U[n + 1, i]

# plotting og animering
fig, ax = plt.subplots(figsize=(8, 4))
plt.subplots_adjust(bottom=0.2)

line, = ax.plot(x, U[0], 'b')
ax.set_xlim(-L, L)
ax.set_ylim(0, u_max)
ax.set_xlabel('Posisjon x')
ax.set_ylabel('Tetthet u(x,t)')
title = ax.set_title(f'Trafikktetthet over tid for p = {p} (t = 0.00 s)')

ax_slider = plt.axes([0.2, 0.05, 0.65, 0.03])
slider = Slider(ax_slider, 'N_t', 0, Nt - 1, valinit=0, valstep=1)

def update(val):
    frame = int(slider.val)
    line.set_ydata(U[frame])
    title.set_text(
        f'Trafikktetthet over tid for p = {p} (t = {frame * dt:.2f} s)')
    fig.canvas.draw_idle()

slider.on_changed(update)
plt.show()
```

4.1.4 Oppgave 2e

```
import numpy as np
import matplotlib.pyplot as plt

L = 1000 # positiv banelengde
v_max = 28 # maksfart i m/s
Nx = 2000 # antall punkt langs den diskretiserte x-aksen
T_max = 600 # maks tid
dx = 2 * L / Nx # delta x mellom hvert punkt
dt = 0.9 * (dx / v_max) # delta t mellom hvert punkt
Nt = int(T_max / dt) # antall tidspunkt
x = np.linspace(-L, L, Nx, dtype=np.float64)
t = np.linspace(0, T_max, Nt)
u_max = 0.1 # maksimal tetthet i biler/m

valid_p = [1, 2, 5]

# hastighetsfunksjon
def v(u, p):
    return v_max * np.power(1 - (u / u_max), p)

# fluksfunksjon
def J(u, p):
    return u * v(u, p)

def compute_traffic(p):
    u = np.zeros(Nx, dtype=np.float64)
    u[x < 0] = u_max # initialverdi

    # initialiser U
    U = np.zeros((Nt, Nx), dtype=np.float64)
    U[0, :] = u.copy()

    # gjennomfør lax-friedrichs metode
    for n in range(Nt - 1):
        J_u = J(u, p)
        for i in range(1, Nx - 1):
            U[n + 1, i] = 0.5 * (u[i - 1] + u[i + 1]) - (dt / (2 * dx)) * (J_u[i + 1] - J_u[i - 1])
        U[n + 1, -1] = 0 # randbetingelse
        u = U[n + 1, :].copy()

    # K(t)
    K = np.zeros(Nt)
    for n in range(Nt - 1):
        first_moving_index = np.where(U[n, :] < u_max)[0][0]
        K[n] = x[first_moving_index]
    return K

results = {}
for p in valid_p:
    K = compute_traffic(p)
    results[p] = K

colors = {1: 'b', 2: 'g', 5: 'r'}

# Plotting
plt.figure()
for p in valid_p:
    plt.plot(t, results[p], color=colors[p], label=f'p = {p}')

plt.xlabel('Tid (s)')
plt.ylabel('Posisjon (m)')
plt.title('Posisjon til bil som begynner å bevege seg ved tid t $K(t)$')
plt.legend()
plt.grid()
plt.show()
```

4.2 Del 2

4.2.1 Oppgave 3

```
# Numerisk løsning av oppgave 3
import numpy as np
import matplotlib.pyplot as plt

antall = 1000
x_verdier = np.linspace(-1, 1, antall)
h = x_verdier[1] - x_verdier[0]
A = np.zeros((antall, antall))
b = np.cos(np.pi * x_verdier)

for i in range(1, antall - 1):
    A[i, i - 1] = 1 / h**2
    A[i, i] = -2 / h**2
    A[i, i + 1] = 1 / h**2

A[0, 0] = 1
b[0] = 0
A[-1, -1] = 1
b[-1] = 2

numerisk_losning = np.linalg.solve(A, b)
analytisk_losning = -1/np.pi**2 * np.cos(np.pi * x_verdier) + x_verdier + 1 - 1/np.pi**2

plt.plot(x_verdier, numerisk_losning, 'ro-', markersize=2, label="Numerisk løsning")
plt.plot(x_verdier, analytisk_losning, 'b-', label="Analytisk løsning")
plt.xlabel("x")
plt.ylabel("Funksjonsuttrykk")
plt.legend()
plt.grid(True)
plt.show()
```

4.2.2 Oppgave 4

```
import numpy as np
import matplotlib.pyplot as plt

minimums_x, maximums_x = -1, 1
storste_t_verdi = 1.0
antall_x = 100
part_deriv_x = (maximums_x - minimums_x) / (antall_x - 1)
part_deriv_t = (0.4 * part_deriv_x**2)
t = int(np.ceil(storste_t_verdi / part_deriv_t))
x = np.linspace(minimums_x, maximums_x, antall_x)
liste_t_verdier = [0, 0.05, 0.075, 1] # Ulike t - verdier for å vise temperaturfordeling over tid

#Varmeligningen
def f(x):
    return np.cos(np.pi * x)

#Startsbetingelsen
def start(x):
    return 1 + x + 5 * np.sin(np.pi * x)

#Rand-betingelsene
u = np.zeros((t + 1, antall_x))
u[0, :] = start(x)
u[:, 0] = 0
u[:, -1] = 2

for n in range(t):
    for i in range(1, antall_x - 1):
        u[n+1, i] = u[n, i] + part_deriv_t *
            ((u[n, i+1] - 2*u[n, i] + u[n, i-1])
             / part_deriv_x**2 - f(x[i]))
    u[n+1, 0] = 0
    u[n+1, -1] = 2

#Lager en for loop for å kunne endre og teste ulike t verdier /
for t_verdier in liste_t_verdier:
    t_tull = min(t, max(0, int(t_verdier / part_deriv_t)))
    plt.plot(x, u[t_tull, :], label=f't={t_verdier:.2f}')

plt.xlabel("x")
plt.ylabel("u(x,t)")
plt.legend()
plt.grid(True)
plt.show()
```

4.2.3 Oppgave 5

```
import numpy as np
import matplotlib.pyplot as plt

antall_x, antall_y = 50, 20
x = np.linspace(-5, 5, antall_x)
y = np.linspace(0, 2, antall_y)
X, Y = np.meshgrid(x, y)
u = np.zeros((antall_x, antall_y))

#Rand-betingelsene
u[0, :] = np.sin(2 * np.pi * y)
u[-1, :] = np.sin(2 * np.pi * y)
u[:, 0] = 0
u[:, -1] = np.sin(np.pi * x)

for i in range(5000):
    u[1:-1, 1:-1] = 0.25 * (u[:-2, 1:-1] + u[2:, 1:-1] + u[1:-1, :-2] + u[1:-1, 2:])

fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X, Y, u.T, cmap="cividis", edgecolor="none")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("u(x, y)")
plt.show()
```

4.2.4 Oppgave 6b

```
# Numerisk løsning
import numpy as np
import matplotlib.pyplot as plt
# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
nx, ny = 40, 28 # Antall punkter i x- og y-retning

dx, dy = Lx / (nx), Ly / (ny) # Romlig steglengde
nx, ny = int(Lx/dx), int(Ly/dy)
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20 # Starttemperatur 20 grader

# Funksjon for å løse varmelikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()

        u[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
        # Randbetingelser
        u[:, 0] = 200 # Venstre kant
        u[:, -1] = 200 # Høyre kant
        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant
    return u

# Løsning og plotting ved ulike tidspunkt
times = [0, 1*60, 2*60, 3*60, 4*60, 5*60] # Tid i sekunder
titles = ["0 minutter", "1 minutter", "2 minutter", "3 minutter", "4 minutter", "5 minutter"]
fig, axes = plt.subplots(3, 2, figsize=(10, 8)) # 3x2 grid av plott
fig.suptitle("Temperaturfordeling i legemet over tid", fontsize=16)

# Løsning og plotting ved ulike tidspunkt
for i, t in enumerate(times):
    steps = int(t / dt)
    u = solve_heat_equation(u, alpha, dx, dy, dt, steps)

    # plotting
    ax = axes[i // 2, i % 2]
    im = ax.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
    ax.set_title(titles[i])
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')

    fig.colorbar(im, ax=ax, label='Temperatur (°C)')

plt.tight_layout()
plt.show()
```

4.2.5 Oppgave 6c

```
import numpy as np
import matplotlib.pyplot as plt

# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
nx, ny = 40, 28 # Antall punkter i x- og y-retning
dx, dy = Lx / (nx - 1), Ly / (ny - 1) # Romlig steglengde
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20

# Funksjon for å løse varmeledningslikningen
def solve_heat_equation2(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        un[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
        # Randbetingelser
        u[:, 0] = 200 # Venstre kant
        u[:, -1] = 200 # Høyre kant
        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant
    return u

# Funksjon for å finne når midten når 60 grader
def find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp):
    steps = 0
    mid_x, mid_y = nx // 2, ny // 2 # Midten av legemet
    while u[mid_x, mid_y] < target_temp:
        steps += 1
        u = solve_heat_equation2(u, alpha, dx, dy, dt, 1) # Oppdater ett tidssteg
    return steps * dt

# Finn tiden
target_temp = 60
time_to_reach = find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp)
print(f"Det tar {time_to_reach/60:.1f} minutter å nå {target_temp}°C i midten.")

# Plot temperaturfordelingen ved dette tidspunktet
steps = int(time_to_reach / dt)
u = solve_heat_equation2(u, alpha, dx, dy, dt, steps)
plt.figure()
plt.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
plt.colorbar(label='Temperatur (°C)')
plt.title(f'Tid: {time_to_reach/60:.1f} minutter')
plt.xlabel('x [m]')
plt.ylabel('y [m]')
plt.show()
```

4.2.6 Oppgave 6d

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
%matplotlib notebook

# Parametere
alpha = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
Lx, Ly = 0.4, 0.28 # Dimensjoner av legemet [m]
nx, ny = 40, 28 # Antall punkter i x- og y-retning
dx, dy = Lx / (nx - 1), Ly / (ny - 1) # Romlig steglengde
dt = dx**2 * dy**2 / (2 * alpha * (dx**2 + dy**2)) # Tidssteg

# Initialbetingelse
u = np.ones((nx, ny)) * 20

# Funksjon for å løse varmeledningslikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        un[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
        # Randbetingelser
        un[:, 0] = 200 # Venstre kant
        un[:, -1] = 200 # Høyre kant
        un[0, :] = 200 # Nedre kant
        un[-1, :] = 200 # Øvre kant
    return u

# Opprett figuren
fig, ax = plt.subplots()
im = ax.imshow(u.T, cmap='bwr', vmin=20, vmax=200)
plt.colorbar(im, label='Temperatur (°C)')
ax.set_xlabel('x [m]')
ax.set_ylabel('y [m]')
ax.set_title('Temperaturfordeling over tid')

# Funksjon for å oppdatere animasjonen
def update(frame):
    global u
    u = solve_heat_equation(u, alpha, dx, dy, dt, 10) # Oppdater 10 tidssteg per frame
    im.set_array(u.T)
    ax.set_title(f'Tid: {frame * dt * 10 / 60:.1f} minutter')
    return im,

# Lag animasjonen
ani = animation.FuncAnimation(fig, update, frames=100, interval=50, blit=True, repeat=False)
plt.show()
```

4.2.7 Oppgave 7a

```
import numpy as np
import matplotlib.pyplot as plt

# Parametere
alpha_steel = 1.172e-5 # Termisk diffusivitet for stål [m^2/s]
alpha_air = 2.0e-5 # Termisk diffusivitet for luft [m^2/s]

Lx, Ly = 0.8, 0.56 # Dimensjoner for hele ovnen (legeme + luft)
L_body_x, L_body_y = 0.4, 0.28 # Dimensjoner av selve legemet

dx, dy = 0.01, 0.01 # Romlig steglengde
nx, ny = int(Lx / dx), int(Ly / dy)

dt = dx**2 * dy**2 / (2 * alpha_air * (dx**2 + dy**2)) # Tidssteg

# Størrelse på legemet
body_x_start, body_x_end = int(0.2 / dx), int((0.2 + L_body_x) / dx)
body_y_start, body_y_end = int(0.14 / dy), int((0.14 + L_body_y) / dy)

# Temperaturmatrise
u = np.ones((nx, ny)) * 200
u[body_x_start:body_x_end, body_y_start:body_y_end] = 15 # Legemet starter på 15°C

alpha = np.ones((nx, ny)) * alpha_air
alpha[body_x_start:body_x_end, body_y_start:body_y_end] = alpha_steel

# Funksjon for å løse varmelikningen
def solve_heat_equation(u, alpha, dx, dy, dt, steps):
    for _ in range(steps):
        un = u.copy()
        u[1:-1, 1:-1] = un[1:-1, 1:-1] + alpha[1:-1, 1:-1] * dt * (
            (un[2:, 1:-1] - 2 * un[1:-1, 1:-1] + un[:-2, 1:-1]) / dx**2 +
            (un[1:-1, 2:] - 2 * un[1:-1, 1:-1] + un[1:-1, :-2]) / dy**2
        )
        # Randbetingelser
        u[:, 0] = 200 # Venstre kant
        u[:, -1] = 200 # Høyre kant
        u[0, :] = 200 # Nedre kant
        u[-1, :] = 200 # Øvre kant
    return u

# Løsning og plotting ved ulike tidspunkt
times = [0, 1*60, 2*60, 3*60, 4*60, 5*60] # Tid i sekunder
titles = ["0 minutter", "1 minutter", "2 minutter", "3 minutter", "4 minutter", "5 minutter"]
fig, axes = plt.subplots(3, 2, figsize=(10, 8)) # 3x2 grid av plott
fig.suptitle("Temperaturfordeling i legemet og luftlaget", fontsize=16)

for i, t in enumerate(times):
    steps = int(t / dt)
    u = solve_heat_equation(u, alpha, dx, dy, dt, steps)

    ax = axes[i // 2, i % 2]
    im = ax.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
    ax.set_title(titles[i])
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')
    fig.colorbar(im, ax=ax, label='Temperatur (°C)')

plt.tight_layout()
plt.show()
```

4.2.8 Oppgave 7b

```
# Oppretter figuren
fig, ax = plt.subplots()
im = ax.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
plt.colorbar(im, label='Temperatur (°C)')
ax.set_xlabel('x [m]')
ax.set_ylabel('y [m]')
ax.set_title('Temperaturfordeling over tid')

# Definerer animasjonen
def update(frame):
    global u
    u = solve_heat_equation(u, alpha, dx, dy, dt, 10) # Oppdater 10 tidssteg per frame
    im.set_array(u.T)
    ax.set_title(f'Tid: {frame * dt * 10 / 60:.1f} minutter')
    return im,

# Lager animasjonen
ani = animation.FuncAnimation(fig, update, frames=100, interval=50, blit=True, repeat=False)
plt.show()
```


4.2.9 Oppgave 7c

```
import numpy as np
import matplotlib.pyplot as plt

# Funksjon for å finne når midten når 60 grader
def find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp, body_x_start, body_x_end,
    ↪ body_y_start, body_y_end):
    steps = 0
    mid_x, mid_y = (body_x_start + body_x_end) // 2, (body_y_start + body_y_end) // 2
    while u[mid_x, mid_y] < target_temp:
        steps += 1
        u = solve_heat_equation2(u, alpha, dx, dy, dt, 1) # Oppdater ett tidssteg
    return steps * dt

# Finn tiden
target_temp = 60
time_to_reach = find_time_to_reach_temp(u, alpha, dx, dy, dt, target_temp, body_x_start,
    ↪ body_x_end, body_y_start, body_y_end)
print(f"Det tar {time_to_reach/60:.1f} minutter å nå {target_temp}°C i midten.")

# Plot temperaturfordelingen ved dette tidspunktet
steps = int(time_to_reach / dt)
u = solve_heat_equation2(u, alpha, dx, dy, dt, steps)
plt.figure()
plt.imshow(u.T, cmap='bwr', vmin=15, vmax=200)
plt.colorbar(label='Temperatur (°C)')
plt.title(f'Tid: {time_to_reach/60:.1f} minutter')
plt.xlabel('x [m]')
plt.ylabel('y [m]')
plt.show()
```